

claude-windows / dbc4cdae-04f6-4d06-af90-397bbce3fc57

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dbc4cdae-04f6-4d06-af90-397bbce3fc57\subagents\agent-a959d05b2001e24e0.jsonl`  
- SHA-256: `e621404218f60164b08cda8290ce6b18be2ac54d9dfe6a650bc307fb1eec2eda`  
- Source modified: `2026-07-06T17:12:15+00:00`  
- Imported at: `2026-07-08T15:59:36+00:00`  
- project: `subagents`  
- session_id: `dbc4cdae-04f6-4d06-af90-397bbce3fc57`
```

Transcript

1. user (2026-07-06T17:10:50.156Z)

Read-only investigation in the repo at C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer. I need to map how video resolution and frame-rate flow into the WASB detection pipeline, and what downsample/proxy knobs already exist (or don't), so I can propose adding a GPU-ffmpeg proxy step (per GitHub issue #483).

Read: config.json (the whole "indexing" section, and any "wasb"/"storage" blocks), backend/config/models.py (look for IndexingConfig, WasbConfig / NativeWasbConfig, StorageConfig and any resolution/fps/proxy fields), backend/worker.py (the processing flow: how it pulls the video, decodes, and calls the segmenter/detector), and backend/pipeline/segmenters/trajectory_hybrid.py. Also grep the whole repo for: "proxy", "downsample", "downscale", "scale_cuda", "NVDEC", "NVENC", "transcode", "fps", "target_fps", "resize", "-vf", "scale=", "ffmpeg", "width", "height", "resolution".

Answer CONCRETELY with file:line and actual config values/defaults:

1. Is there ANY existing step that transcodes/generates a lower-res or lower-fps PROXY of the input before detection? If yes, where and is it default-on? (I believe #483 is NOT yet implemented ? confirm.)
2. What config fields (config.json + models.py) currently control input resolution / fps / any proxy for detection? Their default values?
3. In backend/worker.py, exactly where in the flow would a GPU-ffmpeg proxy-transcode step be inserted (between the GCS video pull and the WASB detect call)? Quote the surrounding lines.
4. After detection on a proxy, how are rally timestamps used to build the final reel ? is the reel built from the ORIGINAL or from whatever was detected on? (I need to confirm timestamps map cleanly back to the original; look for the stitch/compile step.)
5. Does config already carry a per-request or per-asset way to pass overrides (e.g. skip_ai_handoff, timeout_sec) that a proxy setting could ride alongside?

Quote code. Do not modify anything. Return a concise findings report.

2. assistant (2026-07-06T17:10:51.736Z)

I'll conduct a thorough read-only investigation of the codebase to map video resolution and frame-rate flow into the WASB detection pipeline. Let me start by examining the key configuration and code files.

3. user (2026-07-06T17:10:52.131Z)

```
1      {  
2          "sport": "badminton",  
3          "ai_provider": "gemini",  
4          "ai_model": "gemini-flash-latest",  
5          "validation": {  
6              "min_resolution_width": 1280,  
7              "min_resolution_height": 720,  
8              "min_fps": 29.9,
```

```

9         "min_blur_threshold": 20.0,
10        "max_scene_cuts_per_minute": 0.5,
11        "relevance": {
12            "court_green_lower": [
13                35,
14                40,
15                40
16            ],
17            "court_green_upper": [
18                85,
19                255,
20                255
21            ],
22            "court_pixels_min_ratio": 0.05
23        }
24    },
25    "indexing": {
26        "default_segmenter": "gemini",
27        "use_gpu": true,
28        "motion_threshold": 0.08,
29        "min_segment_duration": 3.0,
30        "dead_time_merge_gap": 2.0,
31        "chunk_duration": 90,
32        "chunk_overlap": 10,
33        "detector_model": "fasterrcnn_resnet50_fpn",
34        "detector_score_threshold": 0.5,
35        "yolo_velocity_threshold": 0.15,
36        "yolo_frame_skip": 3,
37        "yolo_tracker": "bytetrack.yaml",
38        "yolo_prefetch_workers": 2,
39        "min_action_window_duration": 2.0,
40        "log_yolo_candidates": true,
41        "store_yolo_candidates": true,
42        "ai_handoff_padding": 2.0,
43        "ai_max_workers": 1,
44        "tracknet": {
45            "wsl_distro": "Ubuntu",
46            "repo_dir": "~/models/TrackNetV4",
47            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48            "conda_env": "TrackNetV4",
49            "weights_path": "",
50            "queue_length": 5,
51            "stage_in_wsl": true,
52            "wsl_stage_dir": "~/clips",
53            "timeout_sec": 1800,
54            "velocity_threshold": 0.02
55        },
56        "wasb": {
57            "wsl_distro": "Ubuntu",
58            "repo_dir": "~/models/WASB-SBDT",
59            "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60            "conda_env": "wasb",
61            "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62            "sport": "badminton",
63            "wsl_stage_dir": "~/clips",
64            "timeout_sec": 1800,
65            "velocity_threshold": 0.02
66        }
67    },
68    "output": {
69        "default_highlights_name": "highlights.mp4",
70        "db_name": "sports_indexer.db"
71    },
72    "stitching": {

```

```

73         "begin_padding": 0.5,
74         "end_padding": 0.5,
75         "use_cache": false,
76         "min_shot_count": 1,
77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         }
85     }
86 }

```

4. user (2026-07-06T17:10:52.614Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer. Did you mean worker?

5. user (2026-07-06T17:10:52.750Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     Unknown / typo'd config keys are rejected at load time via extra="forbid".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         pts_max_gap_seconds: float = Field(
31             default=10.0,
32             ge=0.0,
33             description=(
34                 "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                 "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                 "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                 "recordings, and other encoders may use different GOP sizes ? tune "
38                 "this threshold for your recording device."
39             ),
40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )

```

```

44
45
46 class TrackNetConfig(BaseModel):
47     wsl_distro: str = "Ubuntu"
48     repo_dir: str = "~/models/TrackNetV4"
49     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50     conda_env: str = "TrackNetV4"
51     weights_path: str = ""
52     queue_length: int = 5
53     stage_in_wsl: bool = True
54     wsl_stage_dir: str = "~/clips"
55     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56     velocity_threshold: float = 0.02
57     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58     # decoded frame cache) are deleted automatically ? they are pure intermediates,
59     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61     keep_frames: bool = False
62     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64     # 0 = no extra floor (the headroom guard still applies).
65     min_free_gb: float = 0.0
66
67
68 class WasbIndexConfig(BaseModel):
69     """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """
75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79     conda_env: str = "wasb"
80     weights_path: str = (
81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
88     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91     device: str = "cuda" # auto | cuda | mps | cpu
92     python_bin: str = (
93         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94     )
95     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98     keep_frames: bool = False
99     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100    # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,

```

```

not warns).
101     # 0 = no extra floor (the headroom guard still applies).
102     min_free_gb: float = 0.0
103     # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104     # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105     # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106     # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107     # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108     # Tri-state:
109     #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110     #                               behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111     #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112     stream_video: Optional[bool] = None
113     # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114     # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115     # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,
no
116     # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117     # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118     batch_size: Optional[int] = None
119     # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120     # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121     prefetch_batches: Optional[int] = None
122
123     @field_validator("batch_size")
124     @classmethod
125     def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
126         # A DataLoader batch must be >=1; reject at config time so a typo can't reach
127         # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
128         # when truthy, so 0 stays "off/parity-default").
129         if v is not None and v < 1:
130             raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
131         return v
132
133     @field_validator("prefetch_batches")
134     @classmethod
135     def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
136         # 0 = off (the default). Reject negatives: the runner passes any truthy value as
137         # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
138         # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
139         if v is not None and v < 0:
140             raise ValueError(
141                 f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"
142             )
143         return v
144
145
146     class FusionConfig(BaseModel):
147         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
148
149         Every default reproduces control behaviour: the fusion segmenter persists the
150         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``

```

```

is
151     true ? the person-cue features, but it does NOT gate the served handoff unless a
152     threshold is raised (`min_crossings` / `min_players` default 0 = OFF). The court
mask
153     is optional: `court_polygon=None` ? Gate B counts every detection (player-count-
only);
154     supplied ? off-court persons (spectators / adjacent courts) are excluded.
155     ""
156
157     # Which trajectory substrate seeds the windows (shuttle stays primary).
158     substrate: Literal["wasb", "tracknet"] = "wasb"
159     # Windowing velocity threshold for the fusion family (the engine reads
160     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
161     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
162     velocity_threshold: float = 0.02
163     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
164     # enabled ? so the default path never imports torch / touches the GPU.
165     cue_flags: dict[str, bool] = Field(default_factory=dict)
166     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
167     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
168     min_crossings: int = Field(default=0, ge=0)
169     # Served Gate B: when the person cue is on, drop windows with median in-court
players
170     # < this. 0 = OFF.
171     min_players: int = Field(default=0, ge=0)
172     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
173     court_polygon: Optional[list[list[float]]] = None
174     # Net line for the person two-sided-motion split (person features only ? the shuttle
175     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
176     net_axis: Literal["x", "y"] = "y"
177     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
178
179
180     class IndexingConfig(BaseModel):
181         default_segmenter: str = "yolo_hybrid"
182         use_gpu: bool = True
183         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
184         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
185         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
186         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
187         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
188         detector_impl: str = "auto"
189         motion_threshold: float = 0.08
190         min_segment_duration: float = 3.0
191         dead_time_merge_gap: float = 2.0
192         chunk_duration: float = 90.0
193         chunk_overlap: float = 10.0
194         detector_model: str = "fasterrcnn_resnet50_fpn"
195         detector_score_threshold: float = 0.5
196         yolo_velocity_threshold: float = 0.15
197         yolo_frame_skip: int = 3
198         yolo_tracker: str = "bytetrack.yaml"
199         yolo_prefetch_workers: int = 2
200         min_action_window_duration: float = 2.0
201         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
202         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
203         # `window_min_split_gap` s ? the most likely rally boundary), so one window can't
swallow
204         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).

```

```

205      # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
206      # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
207      # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
208      max_window_duration: float = 6.0
209      window_min_split_gap: float = (
210          0.5 # min internal gap (s) that qualifies as a split point
211      )
212      # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
213      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
214      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
215      # Only cuts (recall can't drop). OFF by default until measured/promoted.
216      window_hard_cap: bool = False
217      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
218      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
219      # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
220      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
221      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
222      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
223      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
224      window_inactivity_end: float = 1.5
225      inactivity_rally_thresh: float = 0.20
226      inactivity_min_density: float = 0.30
227      # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
228      # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
229      # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
230      # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
231      # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
232      # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
233      # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
234      # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
235      # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
236      inactivity_temporal_density: bool = True
237      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
238      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
239      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
240      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
241      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
242      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap

```

```

243         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
244         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
245         window_blob_fallback: bool = False
246         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
247         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
248         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
249         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
250         window_blob_factor: float = 4.0
251         log_yolo_candidates: bool = True
252         store_yolo_candidates: bool = True
253         ai_handoff_padding: float = 2.0
254         ai_max_workers: int = 5
255         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
256         # chunks of a high-bitrate (4K) source blow past the provider upload cap
257         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
258         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
259         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
260         # Matches gemini_refine's --clip-scale-width default.
261         ai_chunk_scale_width: int = 1280
262         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
263         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
264         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
265         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
266         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
267         skip_ai_handoff: bool = False
268         # Optional debug knob: trim every video to the first N seconds before processing.
269         debug_duration: Optional[float] = None
270
271         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
272         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
273         fusion: FusionConfig = Field(default_factory=FusionConfig)
274
275         @field_validator("default_segmenter")
276         @classmethod
277         def segmenter_must_be_registered(cls, v: str) -> str:
278             # Lazy import: the segmenter registry imports modules that may import
279             # config, so we resolve it at validation time rather than at module load.
280             from backend.pipeline.segmenters.base import segmenter_registry
281
282             if segmenter_registry.get(v) is None:
283                 available = sorted(segmenter_registry.list_available().keys())
284                 raise ValueError(
285                     f"Segmenter '{v}' is not registered. Available: {'', '.join(available)}"
286                 )
287             return v
288
289         @model_validator(mode="after")
290         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
291             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
292             # step makes `while t < duration: t += step` loop forever, growing memory before
any
293             # GPU work. Reject the bad config at load time instead.
294             if self.chunk_overlap >= self.chunk_duration:
295                 raise ValueError(
296                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration

```



```

"
297         f"({self.chunk_duration}); otherwise chunking never advances."
298     )
299     return self
300
301
302 class OutputConfig(BaseModel):
303     default_highlights_name: str = "highlights.mp4"
304     db_name: str = "sports_indexer.db"
305     # Directory where the DB, highlight reels, and processed clips are written.
306     # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
307     output_dir: str = "output"
308
309
310 class WatermarkConfig(BaseModel):
311     """Branding overlay burned into each highlight clip during the per-clip re-encode
312     (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
313     (under `stitching.watermark` in config.json) to turn it off. The text is fully
314     configurable. The concat stage stays stream-copy (-c copy)."""
315
316     enabled: bool = True
317     text: str = "Generated by Khelsutra.guru"
318     logo_path: str = "" # optional transparent PNG overlay; "" = no logo
319     font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
320     font: str = "Sans" # fontconfig family used when font_path is empty
321     font_size_ratio: float = Field(
322         default=0.035, gt=0.0, le=0.5
323     ) # text height as a fraction of frame height
324     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
325     box: bool = True # faint backing box behind the text for legibility
326     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
327     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
328         "bottom-right"
329     )
330
331
332 class StitchingConfig(BaseModel):
333     begin_padding: float = 0.5
334     end_padding: float = 0.5
335     use_cache: bool = False
336     min_shot_count: int = 1
337     # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
338     # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
339     # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
340     # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
341     # local detector over-fires and its false positives are mostly SHORT (motion blips,
342     # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
343     # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
344     min_rally_duration: float = Field(default=0.0, ge=0.0)
345     watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
346
347
348 class LoggingConfig(BaseModel):
349     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
350
351     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
352     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
353     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
354     # them to WARNING; set true to restore full chatter for request-flow debugging.
355     verbose_http: bool = False

```

```

356
357
358     class SecurityConfig(BaseModel):
359         # When set, /api/process video_path must resolve under this directory (hosted/tenant
360         # mode). Default None preserves the single-user local 'any local file' workflow.
361         allowed_video_root: Optional[str] = None
362
363         # --- Chunked upload (B2, PR-2) ---
364         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
365         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
366         # contain upload_dir or /api/process will reject the uploaded paths.
367         upload_dir: str = "output/uploads"
368         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
369         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
370         max_upload_mb: int = 4096
371         max_part_mb: int = 95
372         allowed_upload_extensions: List[str] = ["mp4", "mov"]
373
374         # --- M9 edge auth (Cloudflare Access, D9) ---
375         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
376         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
377         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
378         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
379         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
380         # application's audience tag. NO secrets here ? these are public identifiers.
381         edge_auth_enabled: bool = False
382         cf_team_domain: str = (
383             "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
384         )
385         cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
386         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
387         # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
388         # OFF ? Cf-Access is a header, not a cookie.)
389
390
391     class StorageConfig(BaseModel):
392         """"Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
393         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
394         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
395         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
396         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
397         (boto3/google auth chains supply credentials). See ``backend/storage/``.""
398
399         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
400         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
401         output_root: str = ""
402         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
403         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
404         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
405         # input name is resolved against ``input_root`` (e.g.

```

```

`input_root`="gs://bucket/videos"`` + "x.mp4"
406     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
`input_backend`
407     # (``"gcs"`` + "bucket/x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the
input
408     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
409     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
410     input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
411     input_root: str = ""
412     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
413     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
414     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
415     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
416     # the legacy flat layout is used unchanged.
417     single_folder_per_video: bool = False
418     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
419     workspace_root: str = ""
420     # Cloud namespace prefix under the root so per-video folders sit tidily beside
421     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
422     workspace_prefix: str = "workspaces"
423     local: dict[str, Any] = Field(default_factory=dict)
424     s3: dict[str, Any] = Field(default_factory=dict)
425     gcs: dict[str, Any] = Field(default_factory=dict)
426     gdrive: dict[str, Any] = Field(default_factory=dict)
427     # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
428     # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
429     gdrive_api: dict[str, Any] = Field(default_factory=dict)
430
431
432     class DeploymentConfig(BaseModel):
433         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
434         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
435
436         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
437         behaves exactly as before this section existed. A profile is opt-in (``config.json``
438         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
439         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
440         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
441         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
442
443         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
444
445         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
446         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
447         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
448         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
449         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
450         # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
451         # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job

```

```

452         # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
453         # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
454         # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
455         max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
456         # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
457         # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
458         # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
459         # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
460         # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
461         remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
462
463
464     class BillingConfig(BaseModel):
465         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
466
467         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
468         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
469         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
470         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
471         inserts a real, calibrated version and points ``pricebook_version`` at it.
472
473         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
474         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
475         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
476
477         enabled: bool = False
478         pricebook_version: str = (
479             "placeholder-v0" # the seeded $0 version; owner flips to a real one
480         )
481         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
482         margin: float = 0.0 # resale markup (?0); 0 = bill at cost
483
484
485     class FlywheelConfig(BaseModel):
486         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
487         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
488         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
489         the owned model climbs toward the D11 ship floor.
490
491         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
492         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
493         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
494         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
```

```

495         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
496         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
497         owner/counsel/GPU scope. Pairs with M9-consent.""
498
499         enabled: bool = False
500         workspace_prefix: str = (
501             "flywheel" # per-video workspace prefix for captured artifacts
502         )
503         shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture
504         min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
505
506
507     class AppConfig(BaseModel):
508         """Root configuration object.
509
510         All runtime configuration should be accessed via an instance of this class
511         (or the loader) rather than raw dict lookups.
512         """
513
514         sport: Sport = "badminton"
515         ai_provider: str = "gemini"
516         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
517         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
518         ai_model: str = "gemini-flash-latest"
519
520         validation: ValidationConfig = Field(default_factory=ValidationConfig)
521         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
522         output: OutputConfig = Field(default_factory=OutputConfig)
523         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
524         security: SecurityConfig = Field(default_factory=SecurityConfig)
525         logging: LoggingConfig = Field(default_factory=LoggingConfig)
526         storage: StorageConfig = Field(default_factory=StorageConfig)
527         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
528         billing: BillingConfig = Field(default_factory=BillingConfig)
529         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
530
531         # Reject unknown keys ? typed config migration is complete (D3).
532         model_config = {
533             "extra": "forbid",
534             "validate_assignment": True,
535         }
536
537
538     # Convenience type alias for callers that want to annotate
539     Config = AppConfig
540

```

6. user (2026-07-06T17:10:52.814Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7     healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8     -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12    the ``<family>-hybrid-<model>`` detector tag in metadata.

```

```

13 - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14 - ``name`` / ``description`` properties (the registry contract).
15 - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16     detection_params extras such as weights_path).
17
18 This base is deliberately NOT registered; only concrete subclasses register.
19 """
20
21 import concurrent.futures
22 import logging
23 import os
24 import time
25 from typing import Any, Dict, List, Optional, Tuple
26
27 import cv2
28
29 from backend.config.models import IndexingConfig
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.segmenters.base import BasePlaySegmenter
32 from backend.providers import provider_registry
33 from backend.sports import sport_registry
34 from backend.utils.paths import output_base
35 from backend.utils.video import extract_video_chunk, get_video_duration
36
37 logger = logging.getLogger(__name__)
38
39
40 def _fmt_ts(seconds: float) -> str:
41     seconds = max(0, int(seconds))
42     h, rem = divmod(seconds, 3600)
43     m, s = divmod(rem, 60)
44     return f"{h:02d}:{m:02d}:{s:02d}"
45
46
47 class TrajectoryHybridSegmenter(BasePlaySegmenter):
48     """Trajectory-detector hybrid: precise candidate rallies + AI semantic
49 boundaries."""
50
51     #: config section under `indexing` (velocity_threshold lives there), the
52     #: output subdir for trajectories, and the metadata detector-tag prefix.
53     family: str = ""
54     #: human-readable label used in log lines.
55     display_label: str = ""
56
57     def _build_runner(
58         self, idx_cfg: IndexingConfig
59     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
60         """Return (runner, detector-specific detection_params extras) for the default
61         (WSL) path."""
62         raise NotImplementedError
63
64     def _build_native_runner(
65         self, idx_cfg: IndexingConfig
66     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
67         """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
68         with a
69         native runner override this; the base refuses with a clear message (M3.5: WASB
70         only)."""
71         raise NotImplementedError(
72             f"detector_impl='native' is not supported for the "
73             f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
74             has a "
75             f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'."
76         )

```

```

73     def _resolve_runner(
74         self, idx_cfg: IndexingConfig
75     ) -> Tuple[DetectorRunner, Dict[str, Any]]:
76         """Resolve the detector runner, honouring the CPU-stub and Linux-native
overrides.
77
78         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
79         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
80         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
81         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
82         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
83         (the WSL runner) so the default production path is unchanged
84         (``docs/DECOUPLED_COMPUTE``).
85         """
86         # Safety net: accept raw dicts from legacy / test callers.
87         if isinstance(idx_cfg, dict):
88             idx_cfg = IndexingConfig(**idx_cfg)
89         impl = (
90             os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.detector_impl or "auto"
91         ).lower()
92         if impl == "stub":
93             from backend.pipeline.detectors.stub_runner import (
94                 StubConfig,
95                 StubDetectorRunner,
96             )
97
98             logger.info(
99                 "%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
100                 self.display_label or "trajectory",
101             )
102             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {
103                 "detector_impl": "stub"
104             }
105         if impl in ("native", "native_wasb", "wasb_native"):
106             logger.info(
107                 "%s: detector_impl=native ? Linux-native runner (no WSL).",
108                 self.display_label or "trajectory",
109             )
110             return self._build_native_runner(idx_cfg)
111         return self._build_runner(idx_cfg)
112
113     @staticmethod
114     def _probe_fps_width(video_path: str) -> tuple:
115         """Return (fps, frame_width) for a video, with sensible fallbacks."""
116         cap = cv2.VideoCapture(video_path)
117         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
118         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
119         cap.release()
120         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
121
122     @staticmethod
123     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
124         """Provider-reported exchange count, else a duration-based estimate.
125
126         One canonical implementation ? the twins had drifted (wasb relied on
127         ``int(None)`` raising into the fallback; same result, by accident).
128         """
129         shot_count = segment.get("shuttle_exchange_count")
130         if shot_count is None:
131             return max(3, int(duration / 0.8))
132         try:

```

```

133         return int(shot_count)
134     except (ValueError, TypeError):
135         return max(3, int(duration / 0.8))
136
137 # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-----
138 def _enrich_candidate_stats(
139     self,
140     cw: Dict[str, Any],
141     points: List[Any],
142     fps: float,
143     frame_width: float,
144     video_path: str,
145     idx_cfg: "IndexingConfig",
146 ) -> Dict[str, Any]:
147     """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
148     returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
149     rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
150     features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
151     return {
152         "peak_velocity": cw["peak_velocity"],
153         "mean_velocity": cw["mean_velocity"],
154         "active_frame_count": cw["active_frame_count"],
155         "track_id_count": cw["track_id_count"],
156     }
157
158 def _select_for_handoff(
159     self,
160     windows: List[Dict[str, Any]],
161     window_stats: List[Dict[str, Any]],
162     idx_cfg: "IndexingConfig",
163 ) -> List[int]:
164     """Indices of the candidate windows that proceed to the AI handoff (and thus may
165     become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
166     ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
167     Persisted candidates are NOT affected ? they remain the full superset for
offline
168     re-scoring."""
169     return list(range(len(windows)))
170
171 def process_video( # type: ignore[override]
172     self,
173     video_id: str,
174     video_path: str,
175     player_pool: Optional[List[str]] = None,
176     max_frames: Optional[int] = None,
177 ) -> List[Dict[str, Any]]:
178     # override-ignore: the ABC declares the Result contract (Success|Failure)
179     # but every live segmenter still returns a bare list ? the documented
180     # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
181     label = self.display_label
182     # --- Sport profile + AI provider wiring (identical across segmenters) ---
183     sport_name = self.config.sport
184     try:
185         sport_profile = sport_registry.get(sport_name)
186     except KeyError as e:
187         logger.error(str(e))
188         return []
189
190     idx_cfg = self.config.indexing
191     skip_ai = bool(idx_cfg.skip_ai_handoff)
192     provider_name = self.config.ai_provider

```



```

193         if skip_ai:
194             model_name = "local-noai"
195             logger.info(
196                 "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
197                 "be emitted as rallies; no %s handoff, no API key needed.",
198                 label,
199                 provider_name,
200             )
201         else:
202             model_name = self.config.ai_model
203             api_key_env_var = f"{provider_name.upper()}_API_KEY"
204             api_key = os.environ.get(api_key_env_var)
205             if not api_key:
206                 from backend.pipeline.segmenters._keyhint import api_key_hint
207
208                 logger.error(
209                     f"{api_key_env_var} environment variable is not set! "
210                     f"To run the {provider_name} handoff, set your API key. "
211                     + api_key_hint(api_key_env_var)
212                 )
213                 return []
214             try:
215                 provider = provider_registry.get(
216                     provider_name, api_key=api_key, model_name=model_name
217                 )
218             except KeyError as e:
219                 logger.error(str(e))
220                 return []
221
222         video_duration = get_video_duration(video_path)
223         if video_duration <= 0:
224             logger.error("Invalid video duration.")
225             return []
226         fps, frame_width = self._probe_fps_width(video_path)
227
228         # --- Runner config + windowing thresholds ---
229         # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
230         # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
231         # it delegates to the subclass _build_runner (the real WSL/native runner).
232         try:
233             runner, runner_params = self._resolve_runner(idx_cfg)
234         except NotImplementedError as e:
235             # e.g. detector_impl="native" for a family without a native runner
236             # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
237             # crashing.
238             logger.error("%s: %s", label, e)
239             return []
240
241         velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
242         merge_gap = idx_cfg.dead_time_merge_gap
243         min_window_duration = idx_cfg.min_action_window_duration
244         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
245         IndexingConfig.
246         max_window_duration = idx_cfg.max_window_duration
247         window_min_split_gap = idx_cfg.window_min_split_gap
248         window_hard_cap = idx_cfg.window_hard_cap
249         window_inactivity_end = idx_cfg.window_inactivity_end
250         inactivity_rally_thresh = idx_cfg.inactivity_rally_thresh
251         inactivity_min_density = idx_cfg.inactivity_min_density
252         inactivity_temporal_density = idx_cfg.inactivity_temporal_density
253         window_blob_fallback = idx_cfg.window_blob_fallback
254         window_blob_factor = idx_cfg.window_blob_factor
255         handoff_padding = idx_cfg.ai_handoff_padding
256         ai_max_workers = idx_cfg.ai_max_workers

```

```

255     log_candidates = idx_cfg.log_yolo_candidates
256     store_candidates = idx_cfg.store_yolo_candidates
257
258     detection_params = {
259         "detector": self.name,
260         "velocity_thresh": velocity_thresh,
261         "merge_gap": merge_gap,
262         "min_action_window_duration": min_window_duration,
263         **runner_params,
264     }
265
266     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
267     ok, msg = runner.healthcheck()
268     if not ok:
269         logger.error("%s detector environment not ready: %s", label, msg)
270         return []
271     logger.info("%s detector env OK: %s", label, msg)
272
273     # --- Phase 2: trajectory -> candidate rally windows ---
274     # Trajectory detectors process the whole video in one pass (they carry
275     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
276     t_start = time.time()
277     out_dir = os.path.join(output_base(self.config), self.family)
278     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
279     if not csv_path:
280         logger.error("%s produced no trajectory; aborting.", label)
281         return []
282
283     points = runner.parse_trajectory_csv(csv_path)
284     logger.info(
285         "Parsed %d trajectory points (%d visible).",
286         len(points),
287         sum(1 for p in points if p.visible),
288     )
289
290     all_candidate_windows = runner.trajectory_to_action_windows(
291         points,
292         fps=fps,
293         frame_width=frame_width,
294         chunk_start=0.0,
295         velocity_thresh=velocity_thresh,
296         merge_gap=merge_gap,
297         min_window_duration=min_window_duration,
298         chunk_index=0,
299         max_window_duration=max_window_duration,
300         min_split_gap>window_min_split_gap,
301         window_hard_cap>window_hard_cap,
302         window_inactivity_end>window_inactivity_end,
303         inactivity_rally_thresh=inactivity_rally_thresh,
304         inactivity_min_density=inactivity_min_density,
305         inactivity_temporal_density=inactivity_temporal_density,
306         window_blob_fallback>window_blob_fallback,
307         window_blob_factor>window_blob_factor,
308     )
309     logger.info(
310         "Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
311         label,
312         time.time() - t_start,
313         len(all_candidate_windows),
314     )
315
316     if log_candidates:
317         for cw in all_candidate_windows:
318             logger.info(
319                 f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "

```

```

320         f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
321         f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}"
322     )
323
324     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
325     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
326     # persistence and the handoff gate below.
327     window_stats = [
328         self._enrich_candidate_stats(
329             cw, points, fps, frame_width, video_path, idx_cfg
330         )
331         for cw in all_candidate_windows
332     ]
333
334     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
335     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
336     if store_candidates:
337         for i, cw in enumerate(all_candidate_windows):
338             candidate_ids[i] = self.db.add_candidate_segment(
339                 video_id,
340                 detector=self.name,
341                 start_time=cw["start"],
342                 end_time=cw["end"],
343                 chunk_index=cw["chunk_index"],
344                 confidence=min(1.0, cw["peak_velocity"]),
345                 stats=window_stats[i],
346                 detection_params=detection_params,
347             )
348
349     if not all_candidate_windows:
350         return []
351
352     # --- Phase 3: AI provider handoff over padded candidate clips ---
353     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
354     # candidates above stay the full superset ? only the served play_segments are
gated.
355     handoff_indices = self._select_for_handoff(
356         all_candidate_windows, window_stats, idx_cfg
357     )
358
359     ai_segments: List[dict] = []
360     if skip_ai:
361         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
362         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
363         # falls back to the duration-based estimate (no AI exchange count).
364         ai_segments = [
365             {
366                 "start_time": all_candidate_windows[wi]["start"],
367                 "end_time": all_candidate_windows[wi]["end"],
368                 "shuttle_exchange_count": None,
369                 "shot_count_confidence": "LocalNoAI",
370                 "_candidate_id": candidate_ids[wi],
371             }
372             for wi in handoff_indices
373         ]
374         logger.info(
375             "Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies
"
376             "(no AI handoff).",

```

```

377         len(ai_segments),
378     )
379 else:
380     handoff_dir = os.path.join(
381         output_base(self.config), f"chunks_{provider_name}_handoff"
382     )
383     os.makedirs(handoff_dir, exist_ok=True)
384     logger.info(
385         "Phase 3: %s validation on %d candidate windows...",
386         provider_name,
387         len(handoff_indices),
388     )
389
390     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
391         w_start = max(0, window["start"] - handoff_padding)
392         w_end = min(video_duration, window["end"] + handoff_padding)
393         w_dur = w_end - w_start
394         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
395         if not extract_video_chunk(
396             video_path, w_start, w_dur, w_path, reencode=True
397         ):
398             return []
399         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
400         segments, metrics = provider.analyze_video(
401             w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
402         )
403         # B2: a provider failure returns [] with metrics["error"] set ? that is
404         # "we don't know", NOT "no rallies in this window". Surface it loudly so
405         # a
406         # partial run isn't mistaken for a complete one (mirrors the gemini
407         segmenter).
408         if metrics.get("error"):
409             logger.warning(
410                 "[Handoff %d] provider error ? window %d skipped: %s",
411                 wi + 1,
412                 wi,
413                 metrics["error"],
414             )
415         valid = []
416         for seg in segments:
417             try:
418                 seg["start_time"] = float(seg["start_time"]) + w_start
419                 seg["end_time"] = float(seg["end_time"]) + w_start
420                 seg["_candidate_id"] = candidate_ids[wi]
421                 valid.append(seg)
422             except (ValueError, TypeError, KeyError) as e:
423                 logger.warning(
424                     "Dropping segment with unparseable timestamps: %s ? %s",
425                     seg,
426                     e,
427                 )
428         try:
429             os.remove(w_path)
430         except OSError:
431             pass
432         return valid
433
434     with concurrent.futures.ThreadPoolExecutor(
435         max_workers=ai_max_workers
436     ) as executor:
437         futures = [
438             executor.submit(
439                 process_candidate_window, wi, all_candidate_windows[wi]
440             )
441             for wi in handoff_indices

```

```

440         ]
441         for future in futures:
442             ai_segments.extend(future.result())
443     try:
444         os.rmdir(handoff_dir)
445     except OSError:
446         pass
447     logger.info(
448         "Phase 3 completed. AI confirmed %d semantic play segments.",
449         len(ai_segments),
450     )
451
452     # --- Phase 4: DB population + candidate linking ---
453     segments_data = []
454     for segment in ai_segments:
455         try:
456             start = float(segment["start_time"])
457             end = float(segment["end_time"])
458             dur = end - start
459             metadata = {
460                 "shot_count": self._shot_count_from(segment, dur),
461                 "shot_count_confidence": segment.get(
462                     "shot_count_confidence", "Unknown"
463                 ),
464                 "detector": f"{self.family}-hybrid-{model_name}",
465             }
466             segment_id = self.db.add_play_segment(
467                 video_id, start, end, None, None, metadata
468             )
469             if segment_id:
470                 candidate_id = segment.get("_candidate_id")
471                 if candidate_id is not None:
472                     self.db.link_candidate_to_segment(candidate_id, segment_id)
473                 segments_data.append(
474                     {
475                         "id": segment_id,
476                         "start_time": start,
477                         "end_time": end,
478                         "duration": dur,
479                         "metadata": metadata,
480                     }
481                 )
482         except Exception as e:
483             logger.warning("Skipping invalid segment struct: %s", e)
484
485     return segments_data
486

```

7. assistant (2026-07-06T17:10:55.103Z)

Now let me search for worker.py and look for grep results on proxy/transcode/ffmpeg/resolution-related patterns:

8. user (2026-07-06T17:10:55.466Z)

No files found

9. user (2026-07-06T17:10:56.907Z)

```

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/.claude/launch.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/.claude/settings.local.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/.vscode/settings.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer/artifacts/a5_floor_smoke/analysis-a5-smoke/manifest.json

```

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/a5_floor_smoke/analysis-a5-smoke/weights.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/appliance_design.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/cloudrun_e2e_validation/halfA_report.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/cloudrun_e2e_validation/halfB_report.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/cloudrun_e2e_validation/_scratch/make_reel.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/cloudrun_lightup/config.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/landing/make_og.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/long_rally_lens_n15.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/multishuttle_142/person_court_polygon.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/multishuttle_142/_patch_key.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/pr245_review.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/rally_tal_gen0/0.1.0/manifest.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/rally_tal_gen0/0.1.0/weights.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/rally_tal_gen0/0.1.1-n15-shadow.20260704/manifest.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/rally_tal_gen0/0.1.1-n15-shadow.20260704/weights.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/so.json
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/artifacts/so.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/annotations/automated_provider.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/annotations/base.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/annotations/file_provider.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/annotations/__init__.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/api/auth.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/api/jobs.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/api/job_worker.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/api/uploads.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/api/__init__.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/billing.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/compute/base.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/compute/cloud_run.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/compute/__init__.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/config/loader.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/config/models.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/config/presets.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/config/startup.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/config/__init__.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/ablation.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/ablation_pdf.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/annotations.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/audio/e1_probe.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/audio/__init__.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/batch.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/calibrate_local.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/calibrate_wasb.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/calibration.py
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/eval/classifier.py

C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer/backend/eval/distill_local.py

10. assistant (2026-07-06T17:10:58.290Z)

Let me search for the main entry point files and worker-related code:

11. user (2026-07-06T17:10:59.867Z)

C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer/backend/main.py

12. user (2026-07-06T17:10:59.954Z)

C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer/backend/api/job_worker.py
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer/backend/api/__pycache__/job_worker.cpython-313.pyc
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer/tests/test_job_worker_loop.py
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer/tests/test_job_worker_module_seam.py
C:\Users\avidu\Projects\khealsutra-guru\badminton-highlight-indexer/tests/__pycache__/test_job_worker_loop.cpython-313-pytest-9.0.3.pyc

13. assistant (2026-07-06T17:11:01.351Z)

Now let me read the main job worker file and search for proxy/transcode patterns:

14. user (2026-07-06T17:11:01.755Z)

```
1      """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
2      P3).
3      2
4      3      Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
5      4      The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
6      5      that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
7      6      re-claimed when due ? with no central scheduler (the DB is the clock).
8      7
9      8      The monkeypatch contract these functions rely on (and that tests depend on):
10     9      every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
11    10     `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
12    11     `backend.main` module object at call-time (`import backend.main as _main`), never as a
13    12     bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
14    13     ...)` still intercepts the inter-function calls (e.g. a parked job arms the `patched`
15    14     `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
16    lazily
17    15     inside the functions also avoids the circular import at module load (main.py imports
18    this
19    16     module; this module must not import main at import time).
20    17     """
21    18
22    19     import json
23    20     import logging
24    21     import os
25    22     import threading
26    23     import traceback
27    24     from concurrent.futures import ThreadPoolExecutor
28    25     from typing import Any, Dict, Optional
29    26
30    27     logger = logging.getLogger(__name__)
31    28
32    29     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
33    30     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
34    31     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
35    32     # already parallelize their AI handoff internally via their own pools.
36    33     # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
37    _cloud_dispatch_possible), a REMOTE
```

```

34     # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35     # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36     # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37     # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38     # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39     #
40     # ? O2 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
41     # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42     # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43     #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44     #   - detector timeout_sec kills hung WSL/GPU calls
45     #   - Cloud Run poll has a max_wait_sec bound
46     #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47     # A future slice should add a per-job wall-clock timeout (the executor itself
48     # cannot enforce timeouts on the Thread).
49     _job_executor: Optional[ThreadPoolExecutor] = None
50     # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51     # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52     # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53     _drain_max_workers: int = 1
54
55
56     def _cloud_dispatch_possible(config: Any = None) -> bool:
57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_get_cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83         return 1

```



```

84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
108         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
138             _maybe_dispatch_remote, which
139             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
140             # the job row for analytics). NULL ? server default = pre-picker behaviour.

```

```

140         compute=job.get("compute"),
141         force=bool(params.get("force")),
142     )
143
144
145 def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152     job self-scheduled and will be re-claimed when `next_poll_at` is due.
153     """
154     import backend.main as _main
155
156     job_id = job["id"]
157     # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158     # original /api/process pipeline). Both record their terminal state the same way;
only process
159     # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160     is_compile = job.get("kind") == "compile"
161     try:
162         if is_compile:
163             result = _main._execute_compile(
164                 config,
165                 db,
166                 job,
167                 progress=lambda stage: db.set_job_progress(job_id, stage),
168             )
169         else:
170             req = _main._job_to_request(job)
171             result = _main._execute_processing(
172                 config,
173                 db,
174                 req,
175                 progress=lambda stage: db.set_job_progress(job_id, stage),
176             )
177         if result.get("video_id"):
178             db.set_job_video_id(job_id, result["video_id"])
179         db.mark_job_done(job_id, result)
180         if not is_compile:
181             _maybe_record_job_cost(
182                 job, result, config, db
183             ) # M8 cost ledger (default-OFF; process jobs only)
184             _maybe_run_flywheel(
185                 job, result, config
186             ) # M11 data-flywheel (process jobs only)
187     except _main.JobWaiting as w:
188         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190         db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191     except Exception as e:
192         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193         db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196 def _maybe_record_job_cost(
197     job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198 ) -> None:

```

```

199         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
202         persist it.
203
204         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
205         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
206         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
207         + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
208         With the placeholder pricebook the cost is 0 regardless.
209
210         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
211         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
212
213         billing_cfg = getattr(config, "billing", None)
214         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
215             return
216         try:
217             usage = (result or {}).get("usage") or {}
218             db.record_job_cost(
219                 job["id"],
220                 usage=usage,
221                 pricebook_version=billing_cfg.pricebook_version,
222                 gpu_type=usage.get("gpu_type"),
223                 compute_backend=(result or {}).get("compute_backend"),
224                 owner_id=job.get("owner_id"),
225                 margin=billing_cfg.margin,
226             )
227             logger.info(
228                 "Recorded cost for job %s (pricebook=%s).",
229                 job["id"],
230                 billing_cfg.pricebook_version,
231             )
232         except Exception as e: # never wedge a completed job on bookkeeping
233             logger.warning(
234                 "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
235             )
236
237
238     def _maybe_run_flywheel(
239         job: Dict[str, Any], result: Dict[str, Any], config: Any
240     ) -> None:
241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244         land), so this captures **nothing** in production. When activated it captures the
consented job's
245         artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248         row (carries ``owner_id``); ``result`` is the pipeline output."""
249
250         flywheel_cfg = getattr(config, "flywheel", None)
251         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252             return

```

```

253     try:
254         from backend import flywheel
255
256         flywheel.run_for_job(job, result, config)
257     except Exception as e: # never wedge a completed job on the flywheel
258         logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260
261 def _drain_due_jobs(config: Any, db: Any) -> int:
262     """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
264
265     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267     a single-worker box keeps making progress with no central timer (the DB is the
clock).
268     Returns the number of jobs that reached a terminal/parked state this tick.
269     """
270     import backend.main as _main
271
272     ran = 0
273     while True:
274         job = db.claim_due_job()
275         if job is None:
276             break
277         ran += 1
278         _main._run_claimed_job(job, config, db)
279     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281     # on the next submit/drain. Best-effort ? never raises into the worker.
282     _main._schedule_next_drain_if_waiting(config, db)
283     return ran
284
285
286 def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288     self-scheduled job resumes with no further client submit. The drain itself re-checks
289     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290     import backend.main as _main
291
292     try:
293         delay = db.seconds_until_next_due()
294     except Exception as e: # pragma: no cover - defensive; never wedge the worker
295         logger.error(f"Could not compute next-due delay: {e}")
296         return
297     if delay is None:
298         return # nothing waiting
299     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303 def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304     """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305     scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
306     monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""

```

```

307     import backend.main as _main
308
309     timer = threading.Timer(
310         delay_sec,
311         lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
312     )
313     timer.daemon = True
314     timer.start()
315
316
317     #: A parked job further out than this is re-checked by a capped wake rather than one
long
318     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
319     _MAX_DRAIN_WAKE_SEC = 60.0
320
321
322     #
-----
323     # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
324     # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart
325     # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the
bucket with
326     # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
327     # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever
328     # re-dispatching (no double GPU spend).
329
330     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
331     _RESUME_POLL_EVERY_SEC = 15.0
332
333
334     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
335         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
336         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,
337         after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
338         thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
339         NEVER re-dispatched from here. Returns the number of resumes started.
340
341         On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
342         exist ? the rows get today's honest terminal failure instead."""
343         rows = db.interrupted_remote_process_jobs()
344         if not rows:
345             return 0
346         if not _cloud_dispatch_possible(config):
347             for job in rows:
348                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
349                 logger.warning(
350                     "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
351                     "dispatch to Cloud Run, so no remote worker can be in flight.",
352                     len(rows),
353                 )
354             return 0
355         # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
356         # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
357         # default target itself is cloud_run (on a default-local server with cloud-override
358         # capability, a NULL row ran in-process ? no remote worker exists to resume).
359         default_is_cloud = (

```

```

360         getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
361     )
362     started = 0
363     for job in rows:
364         # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
365         # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
366         requested = (str(job.get("compute") or "").strip().lower()
367         if requested != "cloud" and not default_is_cloud:
368             db.mark_job_failed(str(job["id"]), "interrupted by server restart")
369             continue
370         t = threading.Thread(
371             target=_resume_one_remote_job,
372             args=(config, db, dict(job)),
373             daemon=True,
374             name=f"resume-{str(job['id'])[:8]}",
375         )
376         t.start()
377         started += 1
378     if started:
379         logger.warning(
380             "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
381             started,
382         )
383     return started
384
385
386 def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
387     """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
388     ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
389     fail the job HONESTLY from what the bucket says. Never dispatches."""
390     from backend import orchestration, storage
391     from backend.infrastructure.execution_policy import (
392         ExecutionPolicy,
393         PolicyTimeout,
394         poll_until,
395     )
396
397     job_id = str(job["id"])
398     video_id = str(job["video_id"])
399     try:
400         raw_params = job.get("params")
401         params = (
402             raw_params
403             if isinstance(raw_params, dict)
404             else json.loads(raw_params or "{}")
405         )
406         if params.get("force"):
407             # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
408             # but a forced job must NEVER complete from the bucket cache ? force
deliberately
409             # bypasses it, and nothing proves the report belongs to the forced run
rather
410             # than an older completed one.
411             db.mark_job_failed(
412                 job_id,
413                 "control plane restarted during a forced reprocess; the cached result "
414                 "cannot be trusted for force=true ? re-submit with force",
415             )

```

```

416         return
417     if (str(job.get("compute") or "").strip().lower() == "local":
418         # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
419         # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
420         db.mark_job_failed(job_id, "interrupted by server restart")
421         return
422     sc = orchestration._storage_settings(config)
423     output_root = ((sc or {}).get("output_root") or "").rstrip("/")
424     if not sc or not output_root:
425         db.mark_job_failed(
426             job_id,
427             "interrupted by server restart (no output bucket configured to recover
from)",
428         )
429         return
430     report_uri = f"{output_root}/outputs/{video_id}/report.json"
431     max_wait = float(
432         getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
433         or 3900.0
434     )
435     policy = ExecutionPolicy(
436         poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
437         op_timeout_sec=120.0,
438         max_wait_sec=max_wait,
439     )
440     db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
441     try:
442         poll_until(
443             lambda: True if storage.exists(report_uri, config=sc) else None,
444             policy,
445             label=f"resume-{job_id[:8]}",
446             logger=logger,
447             # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
448             on_tick=lambda elapsed: db.set_job_progress(
449                 job_id,
450                 "recovering: waiting for the in-flight cloud run ? "
451                 f"{int(elapsed) // 60} min",
452             ),
453         )
454     except PolicyTimeout:
455         db.mark_job_failed(
456             job_id,
457             "control plane restarted mid-run and the cloud worker's result never "
458             "appeared within the poll budget; re-submit the video (a finished result
"
459             "re-hydrates instantly)",
460         )
461         return
462     payload = orchestration.cached_process_result(
463         db, config, video_id, str(job.get("video_path") or "")
464     )
465     if payload is not None:
466         db.mark_job_done(job_id, payload)
467         logger.warning(
468             "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
469             job_id,
470             video_id,
471         )
472         return
473     # The report exists but did not realize into a usable result ? surface its
verdict.

```

```

474         status = "unreadable"
475         try:
476             report = orchestration._read_bucket_json(report_uri, sc) or {}
477             status = str(report.get("status") or "unreadable")
478         except Exception: # noqa: BLE001 ? the failure message is best-effort
479             pass
480         db.mark_job_failed(
481             job_id,
482             f"cloud worker finished without a usable result after a control-plane
restart "
483             f"(report status: {status}); re-submit the video",
484         )
485     except Exception as e: # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
486         logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
487         try:
488             db.mark_job_failed(job_id, f"restart recovery failed: {e}")
489         except Exception: # noqa: BLE001
490             pass
491

```

15. user (2026-07-06T17:11:02.609Z)

```

Found 236 files
docs\README.md
docs\ALPHA_TESTER_ONBOARDING_FAQ.md
docs\CONTROL_PLANE_DURABLE_REBUILD.md
docs\ALPHA_LAUNCH_READINESS.md
backend\ui\index.html
backend\ui\assets\index-dvIzm83F.css
backend\ui\assets\index-503c-FCB.js
tests\test_compute_backend.py
backend\worker\__main__.py
backend\orchestration.py
backend\main.py
backend\infrastructure\database_jobs.py
backend\api\job_worker.py
backend\api\uploads.py
backend\infrastructure\execution_policy.py
tests\test_api_endpoints.py
backend\config\models.py
docs\NEXT_STEPS.md
docs\ALPHA_LAUNCH_REVIEW_2026-07-04.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
deploy\cloudrun\README.md
deploy\cloudrun\Dockerfile
docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
training\gen0\harness.py
tests\test_served_gate_a_regression.py
tests\test_native_wasb_runner.py
tests\test_golden_manifest.py
tests\test_fusion_golden_telemetry.py
tests\test_fusion_compare.py
tests\test_config_wasb.py
tests\test_audio_features.py
tests\test_calibrate_local.py
tests\test_ablation.py
docs\data_pipeline\GOLDEN_VIDEOS.md
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\detectors\native_wasb_runner.py
backend\eval\ablation_pdf.py
backend\eval\ablation.py
backend\eval\calibrate_local.py
backend\eval\distill_local.py
backend\eval\fusion_audio.py

```


backend\eval\fusion_golden.py
backend\eval\rally_seg_eval.py
backend\eval\rally_seq_proto.py
backend\eval\serve_contrast.py
backend\eval\served_gate_a.py
SESSION_HANDOFF.md
backend\pipeline\stitcher\compiler.py
tests\test_compiler.py
docs\archives\past_projects\AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
.github\workflows\ci.yml
docs\DATA_IN_GCS.md
backend\infrastructure\database.py
docs\CODE_MAP.md
scripts\doctor.py
docs\BACKLOG.md
deploy\digitalocean\bootstrap.sh
docs\HOSTING_PLAN.md
docs\archives\past_projects\KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md
docs\archives\past_projects\CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md
docs\PLATFORM_ARCHITECTURE.md
docs\DESKTOP_APP_PLAN.md
backend\storage\gdrive_api.py
backend\pipeline\segmenters\yolo_hybrid.py
tests\test_video.py
tests\test_ffmpeg_timeouts.py
tests\test_ffmpeg_resolver.py
tests\test_annotation_proxy.py
backend\utils\video.py
backend\utils\ffmpeg.py
backend\utils\annotation_proxy.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\format.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\audio\hit_detector.py
tests\test_api_annotations.py
CLAUDE.md
tests\test_tracknet_runner.py
docs\data_pipeline\VIDEO_RECORDING_GUIDELINES.md
docs\I18N_PLAN.md
docs\QUALITY_ITERATIONS.md
deploy\gcp\README.md
backend\pipeline\detectors\tracknet_runner.py
README.md
tests\test_windowing.py
tests\test_rally_seg_eval.py
backend\eval>windowing.py
backend\pipeline\detectors\rally_gate.py
backend\pipeline\segmenters\trajectory_hybrid.py
docs\NIGHTLY_ENGINE_REGRESSION.md
docs\RALLY_DETECTION_CODE_REVIEW.md
tests\test_rally_gate.py
tests\test_watermark_i18n.py
tests\test_watermark.py
tests\test_worker.py
tests\test_yolo_hybrid.py
tools\cleanup_caches.py
tools\generate_highlights.py
tests\test_validators_base.py
tests\test_training_gen0.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_tracknet_hybrid.py
tests\test_stub_runner.py
tests\test_serve_contrast.py
tests\test_run_telemetry.py

tests\test_rally_seg_eval_report.py
tests\test_rally_seq_proto.py
tests\test_rally_slicer.py
tests\test_reporting_harvest.py
tests\test_reporting.py
tests\test_person_detector.py
tests\test_hit_detector.py
tests\test_motion_segmenter.py
tests\test_nightly_reelcount.py
tests\test_output_base.py
tests\test_p2b_install.py
tests\test_golden_real_fixtures.py
tests\test_experiment.py
tests\test_export_wasb_segments.py
tests\test_fusion_hybrid.py
tests\test_gemini.py
tests\test_generate_highlights.py
tests\test_geometry.py
tests\test_court_gate.py
tests\test_device_context.py
tests\test_distill_local.py
tests\test_eval_batch.py
tests\test_eval_partial_labels.py
tests\test_billing.py
tests\test_bind_and_host.py
tests\test_calibrate_wasb.py
tests\test_cleanup_caches.py
tests\test_annotation_providers.py
scripts\nightly_email.py
backend\utils\geometry.py
backend\utils\run_telemetry.py
backend\tools\export.py
backend\tools\rally_slicer.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\scene_cut.py
backend\providers\gemini.py
backend\reporting\harvest.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\detectors\court_gate.py
backend\pipeline\detectors\fusion_features.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\personalization\serving.py
backend\eval\score_calibration.py
backend\features\rally_seq.py
backend\eval\audio\e1_probe.py
backend\eval\batch.py
backend\eval\calibrate_wasb.py
backend\eval\eval_partial_labels.py
backend\eval\export_wasb_segments.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\eval\harness.py
backend\config\loader.py
backend\annotations\file_provider.py
docs\DECISION_SNAPSHOT_REGRESSION.md
backend\eval\gemini_refine.py
eval_baselines\fixtures\occlusion_break\provenance.json
eval_baselines\fixtures\multi_rally_with_deadtime\provenance.json
eval_baselines\fixtures\manifest.json
eval_baselines\fixtures\fx_r06\provenance.json
eval_baselines\fixtures\fx_r05\provenance.json
eval_baselines\fixtures\fx_r04\provenance.json
eval_baselines\fixtures\fx_r03\provenance.json

eval_baselines\fixtures\fx_r02\provenance.json
eval_baselines\fixtures\fx_r01\provenance.json
eval_baselines\fixtures\clean_single_rally\provenance.json
eval_baselines\fixtures\README.md
docs\data_pipeline\GOLDEN_SET_PHASE1_VIDEOS.md
docs\archives\roora-vendor\R00RA_LABELING_GUIDELINES.md
docs\archives\roora-vendor\GOLDEN_SET_VENDORING.md
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\README.md
docs\archives\past_projects\DECOUPLED_COMPUTE\HANDOFF.md
docs\archives\past_projects\DECOUPLED_COMPUTE\DEPLOY_REPORT_GCP_2026-06-20.md
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md
docs\archives\past_projects\DECOUPLED_COMPUTE\07-COMPUTE-ABSTRACTION.md
docs\archives\past_projects\DECOUPLED_COMPUTE\04-BUILD-AND-TEST-PLAN.md
docs\archives\past_projects\DECOUPLED_COMPUTE\02-COMPUTE-MAP-AND-CONTRACT.md
docs\archives\past_projects\DECOUPLED_COMPUTE\01-EXISTING-PLANS-AND-GAP.md
docs\archives\field-pmf\PMF_MARKET_RESEARCH.md
docs\archives\field-pmf\FIELD_VISIT_PLAYBOOK.md
docs\archives\decisions\DECISIONS.md
docs\UI_PROPOSAL.md
docs\SETUP_NEW_MACHINE.md
docs\ROADMAP.md
docs\REAL_FOOTAGE_VALIDATION_REPORT.md
docs\RALLY_QUALITY_RESEARCH.md
docs\R2_EVAL_METRIC_DESIGN.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\RALLY_DETECTION_QUALITY_REPORT.md
docs\PROMINENT_COURT_DETECTION.md
docs\PACKAGING_PLAN.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\CREATING_REELS.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_truly-local_TEMPLATE.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\README.md
docs\COMPUTE_DECOUPLED_SERVING\architecture.svg
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
docs\COMPUTE_DECOUPLED_SERVING\README.md
docs\COMMERCIALIZATION.md
docs\CLOUD_GPU_DRYRUN_GUIDE.md
deploy\digitalocean\mount_scratch.sh
backend\pipeline\validators__init__.py
backend\assets\fonts\README.md
backend\api\static\app.js
docs\VIDEO_LOCALITY_MODEL.md
docs\STORAGE_SHARING_MODEL.md
docs\PERSONALIZATION_PLAN.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\EXECUTION_POLICY.md
docs\ANALYZERS_AND_RECONCILERS.md
config.json
backend\reporting\spec.yaml
.coveragerc
docs\mockups\ui_alpha_mockup.html
eval_baselines\gemini_wrapper_2026-06-10.json
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
backend\pipeline\audio__init__.py
backend\api\static\index.html
backend\api\static\style.css

16. assistant (2026-07-06T17:11:05.047Z)

Now let me search for specific keywords related to proxy, transcode, and resolution in key files:

17. user (2026-07-06T17:11:06.201Z)

```
backend\main.py:533:# In-flight annotation-proxy generations ? dedup so a browser's many Range
requests to /api/source
backend\main.py:534:# don't each kick off a transcode.
backend\main.py:535:_proxy_inflight: "set[str]" = set()
backend\main.py:536:_proxy_lock = threading.Lock()
backend\main.py:564:     """Fetch a remote source once into an output-scoped source/proxy
cache."""
backend\main.py:591:def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
backend\main.py:592:     """Background, best-effort: generate the annotation proxy ONCE so
subsequent /api/source loads
backend\main.py:595:     with _proxy_lock:
backend\main.py:596:         if video_id in _proxy_inflight:
backend\main.py:598:             _proxy_inflight.add(video_id)
backend\main.py:600:             from backend.utils.annotation_proxy import make_annotation_proxy,
needs_proxy
backend\main.py:602:             if not needs_proxy(src_local):
backend\main.py:603:                 return # already light enough ? no proxy worth making
backend\main.py:604:             make_annotation_proxy(
backend\main.py:608:                 logger.warning("proxy: warm failed for %s: %s", video_id, e)
backend\main.py:610:                 with _proxy_lock:
backend\main.py:611:                     _proxy_inflight.discard(video_id)
backend\main.py:617: frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled +
seekable, labels still keyed
backend\main.py:618: to the master's MD5) once one exists; otherwise serves the source and
WARMS a proxy in the
backend\main.py:626:     # 1) A ready proxy wins. Bare-name + containment guarded like
/api/highlights.
backend\main.py:628:         proxy_name = safe_output_filename(f"{video_id}.mp4")
backend\main.py:629:         proxy_path = resolve_under_root(
backend\main.py:630:             os.path.join(_proxies_dir(cfg), proxy_name), _proxies_dir(cfg)
backend\main.py:633:         proxy_path = ""
backend\main.py:634:         if proxy_path and os.path.isfile(proxy_path):
backend\main.py:635:             return FileResponse(proxy_path, media_type="video/mp4")
backend\main.py:665:     # 3) Warm a proxy in the background for next time.
backend\main.py:667:         target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True
backend\main.py:1103: can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing
it synchronously returned a
backend\main.py:1628:             f"gate (edge_auth_enabled / a reverse proxy) is in
front, and set "
backend\api\static\app.js:374:             // ffmpeg stitch can run minutes ? longer than the
edge/proxy timeout ? so it can't block
backend\reporting\spec.yaml:56:         re-encoded/downscaled before upload (0 disables re-encoding
and reintroduces
backend\worker\_main_.py:294:             "detections, or the input resolution differs from
the tuned proxy resolution. "
backend\worker\_main_.py:369:         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy``
(``_proxy`` is part of the name, kept)
backend\providers\gemini.py:133:             f"Re-encode/downscale chunks
(indexing.ai_chunk_scale_width) or chunk smaller."
backend\eval\fusion_golden.py:64:     The collector records source/proxy ownership and full-file
MD5s. Only entries marked
backend\eval\fusion_golden.py:96:     for local source/proxy video location and overrides
derived/stale manifest paths.
backend\eval\fusion_golden.py:439:         "collector golden_source_videos.json; overrides
video paths for source/proxy "
backend\eval\fusion_audio.py:161:         "collector golden_source_videos.json; overrides
video paths for source/proxy "
```

```

backend\eval\gemini_refine.py:227:      """Full-context pass: hand Gemini the WHOLE clip
(downscaled, chunked to stay under the
backend\eval\gemini_refine.py:333:      help="downscale uploaded clips to this width (0=keep
source res)",
backend\eval\gemini_refine.py:338:      help="full-context pass: send the whole clip
(chunked/downscaled) to Gemini "
backend\utils\annotation_proxy.py:1:"""Frame-preserving annotation proxy
(STUDIO_MEDIA_LIFECYCLE.md D4).
backend\utils\annotation_proxy.py:3:A big 4K master is heavy to decode + scrub in a browser. For
in-UI annotation we serve a downscaled
backend\utils\annotation_proxy.py:4:proxy that keeps the SAME timeline (fps + duration) ? so a
rally marked at t seconds on the proxy is
backend\utils\annotation_proxy.py:6:dropped; duration parity is VERIFIED after the transcode,
and a proxy whose timeline drifted is
backend\utils\annotation_proxy.py:28:#: Max proxy height (px). A source already at/below this
needs no proxy (serving it is already light).
backend\utils\annotation_proxy.py:48:      "proxy: ffprobe height probe timed out after %s
for %s",
backend\utils\annotation_proxy.py:54:      logger.warning("proxy: could not probe height for
%s: %s", path, e)
backend\utils\annotation_proxy.py:58:def needs_proxy(src: str, max_height: int =
DEFAULT_MAX_HEIGHT) -> bool:

```

[Showing results with pagination = limit: 50]

18. user (2026-07-06T17:11:06.927Z)

```

backend\main.py:1178:      fps = cap.get(cv2.CAP_PROP_FPS)
backend\ui\index.html:5:      <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
backend\pipeline\validators\relevance.py:82:      small_frame = cv2.resize(frame, (320,
240))
backend\pipeline\validators\scene_cut.py:28:      fps = cap.get(cv2.CAP_PROP_FPS)
backend\pipeline\detectors\person_detector.py:207:      fps = cap.get(cv2.CAP_PROP_FPS)
backend\pipeline\detectors\person_detector.py:211:      frame_width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH)
backend\pipeline\detectors\person_detector.py:335:      fps = cap.get(cv2.CAP_PROP_FPS)
backend\pipeline\detectors\person_detector.py:338:      frame_width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH)
backend\pipeline\detectors\person_detector.py:339:      frame_height =
cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
backend\pipeline\detectors\person_detector.py:387:      fps = cap.get(cv2.CAP_PROP_FPS) if
cap.isOpened() else 0.0
backend\pipeline\detectors\person_detector.py:390:      fw = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
if cap.isOpened() else 0.0
backend\pipeline\detectors\person_detector.py:391:      fh =
cap.get(cv2.CAP_PROP_FRAME_HEIGHT) if cap.isOpened() else 0.0
backend\utils\ffmpeg.py:9:DEFAULT (env unset) returns the bare name (`"ffmpeg"` /
`"ffprobe"`) ? PATH resolution, exactly
backend\pipeline\segmenters\motion.py:65:      fps = cap.get(cv2.CAP_PROP_FPS)
backend\pipeline\segmenters\trajectory_hybrid.py:117:      fps = cap.get(cv2.CAP_PROP_FPS) if
cap.isOpened() else 0.0
backend\pipeline\segmenters\trajectory_hybrid.py:118:      width =
cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
backend\utils\annotation_proxy.py:74:      Only the resolution is scaled (`scale=-2:H`);
fps/duration are untouched and duration parity is
backend\utils\annotation_proxy.py:89:      "-vf", f"scale=-2:{max_height}",
backend\utils\video.py:81:      cmd += ["-vf", f"scale={int(scale_width)}:-2"]
backend\ui\assets\index-dvIzm83F.css:1:[Omitted long matching line]
backend\api\static\index.html:5:      <meta name="viewport" content="width=device-width, initial-
scale=1.0">
backend\ui\assets\index-503c-FCB.js:48:[Omitted long matching line]
backend\ui\assets\index-503c-FCB.js:49:[Omitted long matching line]
backend\ui\assets\index-503c-FCB.js:57:[Omitted long matching line]
backend\ui\assets\index-503c-FCB.js:60:[Omitted long matching line]

```

19. assistant (2026-07-06T17:11:08.696Z)

Let me now read the ffmpeg and video utility files, plus the WASB runner and annotation proxy:

20. user (2026-07-06T17:11:09.377Z)

```
1     import logging
2     import subprocess
3     from typing import Optional
4
5     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
6         ffmpeg_bin,
7         ffmpeg_timeout_sec,
8         ffprobe_bin,
9         ffprobe_timeout_sec,
10        timeout_label,
11    )
12
13    logger = logging.getLogger(__name__)
14
15
16    def get_video_duration(video_path: str) -> float:
17        """Returns the exact duration of a video file in seconds using ffprobe.
18        Returns 0.0 on failure."""
19        try:
20            timeout = ffprobe_timeout_sec()
21            duration_output = (
22                subprocess.check_output(
23                    [
24                        ffprobe_bin(),
25                        "-v",
26                        "error",
27                        "-show_entries",
28                        "format=duration",
29                        "-of",
30                        "default=noprint_wrappers=1:nokey=1",
31                        video_path,
32                    ],
33                    timeout=timeout,
34                )
35                .decode()
36                .strip()
37            )
38            return float(duration_output)
39        except subprocess.TimeoutExpired:
40            logger.error(
41                "Timed out detecting video duration for %s after %s",
42                video_path,
43                timeout_label(ffprobe_timeout_sec()),
44            )
45            return 0.0
46        except Exception as e:
47            logger.error(f"Could not detect video duration for {video_path}: {e}")
48            return 0.0
49
50
51    def extract_video_chunk(
52        video_path: str,
53        start_time: float,
54        duration: float,
55        output_path: str,
56        reencode: bool = False,
57        scale_width: Optional[int] = None,
58    ) -> bool:
59        """Extracts a chunk from a video file.
```

```

60
61     Args:
62         reencode: If True, re-encodes with libx264 (slower but frame-accurate).
63                   If False, uses stream-copy (fast but cuts at nearest keyframe).
64                   Use True for short clips where keyframe drift matters.
65         scale_width: If set (requires reencode), downscale to this width (height auto,
66                     even). Useful for high-res sources headed to an API that analyses at
67                     low res anyway ? shrinks uploads with no analysis loss.
68     """
69     cmd = [
70         ffmpeg_bin(),
71         "-y",
72         "-ss",
73         str(start_time),
74         "-t",
75         str(duration),
76         "-i",
77         video_path,
78     ]
79     if reencode:
80         if scale_width:
81             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
82             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
83         else:
84             cmd += ["-c", "copy"]
85     cmd.append(output_path)
86     try:
87         timeout = ffmpeg_timeout_sec()
88         subprocess.run(
89             cmd,
90             check=True,
91             stdout=subprocess.DEVNULL,
92             stderr=subprocess.DEVNULL,
93             timeout=timeout,
94         )
95         return True
96     except subprocess.TimeoutExpired:
97         logger.error(
98             "Timed out extracting chunk %.3fs - %.3fs after %s",
99             start_time,
100             start_time + duration,
101             timeout_label(ffmpeg_timeout_sec()),
102         )
103         return False
104     except Exception as e:
105         logger.error(
106             f"Failed to extract chunk {start_time}s - {start_time + duration}s: {e}"
107         )
108         return False
109

```

21. user (2026-07-06T17:11:09.511Z)

```

1     """Frame-preserving annotation proxy (STUDIO_MEDIA_LIFECYCLE.md D4).
2
3     A big 4K master is heavy to decode + scrub in a browser. For in-UI annotation we serve a
4     downscaled
5     proxy that keeps the SAME timeline (fps + duration) ? so a rally marked at t seconds on
6     the proxy is
7     t seconds on the master, and the labels still key to the master's file MD5. Only
8     resolution is
9     dropped; duration parity is VERIFIED after the transcode, and a proxy whose timeline
10    drifted is
11    discarded (the caller falls back to serving the source). Best-effort + stdlib-only;
12    never raises.

```

```

8     """
9     from __future__ import annotations
10
11     import logging
12     import os
13     import subprocess
14     import tempfile
15
16     from backend.utils.ffmpeg import (
17         ffmpeg_available,
18         ffmpeg_bin,
19         ffmpeg_timeout_sec,
20         ffprobe_bin,
21         ffprobe_timeout_sec,
22         timeout_label,
23     )
24     from backend.utils.video import get_video_duration
25
26     logger = logging.getLogger(__name__)
27
28     #: Max proxy height (px). A source already at/below this needs no proxy (serving it is
already light).
29     DEFAULT_MAX_HEIGHT = 720
30     #: Duration must match the source within this (s), else the timeline drifted ? discard
(protects the
31     #: second-based marks the annotator produces).
32     _DURATION_TOLERANCE_S = 0.5
33
34
35     def probe_height(path: str) -> int:
36         """The video's height in px, or 0 on failure."""
37         try:
38             timeout = ffprobe_timeout_sec()
39             out = subprocess.check_output(
40                 [ffprobe_bin(), "-v", "error", "-select_streams", "v:0", "-show_entries",
41                 "stream=height", "-of", "default=noprint_wrappers=1:nokey=1", path],
42                 text=True,
43                 timeout=timeout,
44             ).strip().splitlines()
45             return int(out[0]) if out and out[0].strip() else 0
46         except subprocess.TimeoutExpired:
47             logger.warning(
48                 "proxy: ffprobe height probe timed out after %s for %s",
49                 timeout_label(ffprobe_timeout_sec()),
50                 path,
51             )
52             return 0
53         except Exception as e: # noqa: BLE001 - probe is best-effort
54             logger.warning("proxy: could not probe height for %s: %s", path, e)
55             return 0
56
57
58     def needs_proxy(src: str, max_height: int = DEFAULT_MAX_HEIGHT) -> bool:
59         """True when ``src`` is tall enough that a downscaled proxy is worth generating."""
60         return probe_height(src) > max_height
61
62
63     def _cleanup(path: str) -> None:
64         try:
65             os.remove(path)
66         except OSError:
67             pass
68
69
70     def make_annotation_proxy(src: str, dst: str, max_height: int = DEFAULT_MAX_HEIGHT) ->

```



```

bool:
71     """Transcode ``src`` ? a downscaled, timeline-preserving proxy at ``dst`` (written
atomically).
72     Returns True on a verified proxy, False otherwise (caller serves the source). Never
raises.
73
74     Only the resolution is scaled (``scale=-2:H``); fps/duration are untouched and
duration parity is
75     verified, so second-based marks transfer to the master unchanged. Audio is kept (the
shuttle
76     'thwack' helps the rater judge rally boundaries)."""
77     if not ffmpeg_available():
78         logger.info("proxy: ffmpeg unavailable; serving source (%s)", src)
79         return False
80     if not os.path.isfile(src):
81         return False
82     src_dur = get_video_duration(src)
83     dst_dir = os.path.dirname(dst) or "."
84     os.makedirs(dst_dir, exist_ok=True)
85     fd, tmp = tempfile.mkstemp(dir=dst_dir, prefix=".proxy-", suffix=".mp4")
86     os.close(fd)
87     cmd = [
88         ffmpeg_bin(), "-y", "-i", src,
89         "-vf", f"scale=-2:{max_height}",
90         "-c:v", "libx264", "-preset", "veryfast", "-crf", "28", "-pix_fmt", "yuv420p",
91         "-g", "48", # denser keyframes ? snappy scrubbing
92         "-c:a", "aac", "-b:a", "96k",
93         "-movflags", "+faststart",
94         tmp,
95     ]
96     try:
97         timeout = ffmpeg_timeout_sec()
98         subprocess.run(cmd, check=True, capture_output=True, timeout=timeout)
99     except subprocess.TimeoutExpired:
100         logger.warning(
101             "proxy: ffmpeg timed out after %s for %s",
102             timeout_label(ffmpeg_timeout_sec()),
103             src,
104         )
105         _cleanup(tmp)
106         return False
107     except Exception as e: # noqa: BLE001 - a transcode failure is non-fatal; fall back
to the source
108         logger.warning("proxy: ffmpeg failed for %s: %s", src, e)
109         _cleanup(tmp)
110         return False
111     proxy_dur = get_video_duration(tmp)
112     if src_dur > 0 and abs(proxy_dur - src_dur) > _DURATION_TOLERANCE_S:
113         logger.warning("proxy: duration parity off for %s (src=%.3fs proxy=%.3fs);
discarding",
114                        src, src_dur, proxy_dur)
115         _cleanup(tmp)
116         return False
117     try:
118         os.replace(tmp, dst)
119     except OSError as e:
120         logger.warning("proxy: could not place proxy at %s: %s", dst, e)
121         _cleanup(tmp)
122         return False
123     logger.info("proxy: wrote annotation proxy %s (%.1fs)", dst, proxy_dur)
124     return True
125

```

Now let me read the WASB runner to see how detection works:

23. user (2026-07-06T17:11:12.002Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
5      M3.5).
6      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
7      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
8      core
9      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
10     tolerance;
11     see the acceptance gate below) ? but WITHOUT the WSL transport:
12
13     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
14     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
15     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
16     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
17     ``auto``
18     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
19     ``cuda`` stays
20     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
21     ``runner.device`` override).
22     * weights / repo / python / scratch come from the environment first
23     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
24     ``indexing.wasb.`` config, then a default ? so a box needs no config edits.
25
26     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__vid12_wasb.csv``
27     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
28     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
29     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
30     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
31     ``detector_impl="native"``.
32
33     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
34     runner
35     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
36     same
37     candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
38     deterministic
39     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
40     exact
41     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
42     streaming
43     vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
44     artifact), and two native runs were byte-identical (deterministic). This is a hardware
45     check
46     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
47     contract,
48     not the numbers.
49     """
50
51     import logging
52     import os
53     import shutil
54     import subprocess
55     from dataclasses import dataclass
56     from typing import List, Optional, Tuple
57
58     from backend.config.models import IndexingConfig
59     from backend.pipeline.detectors.base import DetectorRunner
60     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
```

```

48     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51     logger = logging.getLogger(__name__)
52
53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63
64     @dataclass
65     class NativeWasbConfig:
66         """Resolved settings for a Linux-native WASB run.
67
68         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
69         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
70         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
71         """
72
73         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
74         python_bin: str = (
75             "python" # the `wasb` conda env python (invoked by path, no activate)
76         )
77         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
78         sport: str = "badminton"
79         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
80         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
81         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
82         keep_frames: bool = (
83             False # retain the decoded frame cache after success (debug only)
84         )
85         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
86         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
87         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
88         stream_video: bool = True
89         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
90         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
91         batch_size: int = 0
92         prefetch_batches: int = 0
93
94         @classmethod
95         def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
96             if isinstance(idx_cfg, dict):
97                 idx_cfg = IndexingConfig(**idx_cfg)
98             w = idx_cfg.wasb
99             env = os.environ.get
100             # stream_video is tri-state in config: None/absent => native default (stream);
an
101             # explicit True/False still wins (e.g. False for a container cv2 can't seek).
102             sv = w.stream_video
103             return cls(

```

```

104         repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
105         python_bin=env("WASB_PYTHON") or w.python_bin,
106         weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
107         sport=w.sport,
108         device=env("WASB_DEVICE") or w.device,
109         scratch_dir=env("WASB_SCRATCH")
110         or env("RALLY_SCRATCH")
111         or env("TMPDIR")
112         or "",
113         timeout_sec=w.timeout_sec,
114         keep_frames=w.keep_frames,
115         stream_video=(True if sv is None else bool(sv)),
116         batch_size=int(w.batch_size or 0),
117         prefetch_batches=int(w.prefetch_batches or 0),
118     )
119
120
121     class NativeWasbRunner(DetectorRunner):
122         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
123         docstring."""
124         def __init__(self, cfg: NativeWasbConfig):
125             self.cfg = cfg
126             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
127             healthcheck;
128             # lazily probed by run_predict if healthcheck was skipped. None = not yet
129             probed.
130             self._cuda_available: Optional[bool] = None
131
132             # --- low-level native exec (the single place tests patch)
133             -----
134             def _run(
135                 self, argv: List[str], cwd: Optional[str] = None
136             ) -> subprocess.CompletedProcess:
137                 logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
138                 return subprocess.run(
139                     argv,
140                     cwd=cwd,
141                     capture_output=True,
142                     text=True,
143                     timeout=(self.cfg.timeout_sec or None),
144                 )
145
146             # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
147             cpu) -----
148             def _probe_cuda(self) -> bool:
149                 """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
150                 Any failure
151                 (missing python / timeout / import error) is treated as 'no CUDA'."""
152                 try:
153                     res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
154                 except (FileNotFoundError, subprocess.TimeoutExpired):
155                     return False
156                 return res.returncode == 0 and "cuda True" in (res.stdout or "")
157
158             def _cuda_is_available(self) -> bool:
159                 """CUDA availability, cached across healthcheck + run_predict so we probe at
160                 most once."""
161                 if self._cuda_available is None:
162                     self._cuda_available = self._probe_cuda()
163                 return self._cuda_available
164
165             def _effective_device(self) -> str:
166                 """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
167                 the CUDA

```

```

161         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
162         if self.cfg.device == AUTO:
163             return DeviceContext(AUTO, self._cuda_is_available()).effective
164         return self.cfg.device
165
166     def _repo_src(self) -> str:
167         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
168
169     def _copy_infer_script(self) -> bool:
170         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
171         repo_src = self._repo_src()
172         try:
173             os.makedirs(repo_src, exist_ok=True)
174             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
175             return True
176         except OSError as e:
177             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
178             return False
179
180     def _rm_rf(self, path: Optional[str]) -> None:
181         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
182         if path:
183             shutil.rmtree(path, ignore_errors=True)
184
185     def healthcheck(self) -> Tuple[bool, str]:
186         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
187         if not self.cfg.weights_path:
188             return False, (
189                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).\"
190             )
191         weights = os.path.expanduser(self.cfg.weights_path)
192         if not os.path.exists(weights):
193             return False, f"WASB weights not found at {weights}\"
194         repo_src = self._repo_src()
195         if not os.path.isdir(repo_src):
196             return False, (
197                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT \"
198                 f\"(set WASB_REPO_DIR / indexing.wasb.repo_dir).\"
199             )
200         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic

```

24. assistant (2026-07-06T17:11:14.018Z)

Let me continue reading the native WASB runner and then look at the stitcher/compiler to see how timestamps are used:

25. user (2026-07-06T17:11:14.884Z)

```

1     import logging
2     import os
3     import subprocess
4     import tempfile
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
9         ffmpeg_bin,
10        ffmpeg_timeout_sec,
11        ffprobe_bin,

```

```

12         ffprobe_timeout_sec,
13         timeout_label,
14     )
15
16     logger = logging.getLogger(__name__)
17
18     # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
19     # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
20     # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
21     # backend/pipeline/stitcher/, so parents[2] == backend/.
22     _BUNDLED_FONT = (
23         Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-Regular.ttf"
24     )
25
26
27     class VideoCompiler:
28         def __init__(
29             self,
30             output_dir: str,
31             end_padding: float = 1.0,
32             begin_padding: float = 0.5,
33             watermark: Optional[Any] = None,
34         ):
35             self.output_dir = output_dir
36             self.end_padding = end_padding
37             self.begin_padding = begin_padding
38             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
39             # normalized to a dict (or None) so this module stays config-package-agnostic.
40             self.watermark = self._normalize_watermark(watermark)
41             os.makedirs(output_dir, exist_ok=True)
42
43         @staticmethod
44         def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
45             if watermark is None:
46                 return None
47             if hasattr(watermark, "model_dump"):
48                 return watermark.model_dump()
49             if isinstance(watermark, dict):
50                 return dict(watermark)
51             return None
52
53         @staticmethod
54         def _ff_quote(path: str) -> str:
55             """Make a filesystem path safe to embed inside single quotes in an ffmpeg
56             filtergraph operand (drawtext fontfile/textfile, movie= source): forward
57             slashes (Windows-friendly), escaped single quotes, AND escaped colons.
58
59             The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
60             option parser treats ':' as the option separator, so a Windows drive-letter
61             path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
62             the encode fails with "Invalid argument". This silently disabled the
63             on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
64             return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
65
66         @staticmethod
67         def _parse_time(val: Union[int, float, str]) -> float:
68             """Normalizes a timestamp value to float seconds.
69             Handles: float/int (pass-through), plain numeric strings ("12.4"),
70             and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
71             This mirrors the parse_time logic used in gemini.py to ensure
72             the stitcher can always consume whatever format the indexer produced."""
73             if isinstance(val, (int, float)):

```

```

74         return float(val)
75     if isinstance(val, str):
76         val = val.strip()
77         if ":" in val:
78             # Handle MM:SS, HH:MM:SS, etc.
79             parts = val.split(":")
80             parts.reverse()
81             total = 0.0
82             for i, p in enumerate(parts):
83                 try:
84                     total += float(p) * (60**i)
85                 except ValueError:
86                     pass
87             return total
88         try:
89             return float(val)
90         except ValueError:
91             logger.warning(
92                 f"Could not parse timestamp value '{val}' as a number. Defaulting to
0.0."
93             )
94             return 0.0
95     logger.warning(
96         f"Unexpected timestamp type {type(val).__name__} for value '{val}'.
Defaulting to 0.0."
97     )
98     return 0.0
99
100 @staticmethod
101 def _merge_overlapping(
102     segments: List[Tuple[float, float]],
103 ) -> List[Tuple[float, float]]:
104     """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
105
106     A highlight reel must never replay the same footage. The same rally can land in
107     the segment list more than once ? e.g. two detector runs accumulated for one
108     video (add_video is an UPSERT, so re-running without a prune leaves both sets),
109     or
110     chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
111     rally
112     twice. Merging any range that overlaps or abuts the previous one collapses true
113     duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
114     Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
115     """
116     parsed = []
117     for raw_s, raw_e in segments:
118         s = VideoCompiler._parse_time(raw_s)
119         e = VideoCompiler._parse_time(raw_e)
120         if e > s:
121             parsed.append((s, e))
122     parsed.sort()
123     merged: List[Tuple[float, float]] = []
124     for s, e in parsed:
125         if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
126             merged[-1] = (merged[-1][0], max(merged[-1][1], e))
127         else:
128             merged.append((s, e))
129     return merged
130
131 def _get_video_duration(self, video_path: str) -> float:
132     """Probes the video file to get its total duration in seconds.
133     Returns 0.0 if it cannot be determined."""
134     try:
135         timeout = ffprobe_timeout_sec()
136         result = subprocess.run(

```

```

135         [
136             ffprompt_bin(),
137             "-v",
138             "error",
139             "-show_entries",
140             "format=duration",
141             "-of",
142             "default=noprint_wrappers=1:nokey=1",
143             video_path,
144         ],
145         capture_output=True,
146         text=True,
147         check=True,
148         timeout=timeout,
149     )
150     return float(result.stdout.strip())

```

26. user (2026-07-06T17:11:15.696Z)

```

55:     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
63:     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS, not
warns).
85:     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
100:    # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS, not
warns).
111:    # True/False = force on/off for BOTH runners (e.g. set False for a container cv2 can't
seek).
244:    # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
267:    skip_ai_handoff: bool = False
441:    (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.

```

27. assistant (2026-07-06T17:11:17.474Z)

Now let me read the main.py to see the processing flow where a video is pulled and where detection happens:

28. user (2026-07-06T17:11:18.006Z)

```

1     import argparse
2     import json
3     import logging
4     import os
5     import shutil
6     import sys
7     import tempfile
8     import threading
9     import uuid
10    from importlib.metadata import PackageNotFoundError
11    from importlib.metadata import version as _pkg_version
12    from urllib.parse import quote
13
14    import uvicorn
15    from dotenv import load_dotenv
16
17    # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
18    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
19    # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
20    # stdlib-only import ? keeps this pre-pydtastic line light.
21    from backend.utils.app_home import env_file_to_load, resolve_output_dir
22
23    load_dotenv(env_file_to_load())

```



```

24
25     # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
26     logging.basicConfig(
27         level=logging.INFO,
28         format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
29         datefmt="%H:%M:%S",
30     )
31     logger = logging.getLogger(__name__)
32     from typing import Any, Dict, List, Optional
33
34     from fastapi import FastAPI, HTTPException, Request
35     from fastapi.middleware.cors import CORSMiddleware
36     from fastapi.responses import (
37         FileResponse,
38         HTMLResponse,
39         JSONResponse,
40         RedirectResponse,
41     )
42     from fastapi.staticfiles import StaticFiles
43     from pydantic import BaseModel
44     from starlette.middleware.trustedhost import TrustedHostMiddleware
45
46     from backend import (
47         billing, # M8: per-job cost ledger (USD internal / INR display)
48         storage, # M2: URI-aware input acquisition (local = identity passthrough)
49     )
50     from backend.api import (
51         auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
52     )
53     from backend.api import (
54         jobs as jobs_api, # /api/jobs list route (router) + submit-dedup helper (#448)
55     )
56     from backend.config import (
57         AppConfig,
58         StartupCheckError,
59         enforce_startup_checks,
60         write_light_default_config, # P2b: batteries-included first-run config seeding
61     )
62     from backend.config import ( # new typed config
63         load_config as load_app_config,
64     )
65     from backend.infrastructure.database import Database
66     from backend.pipeline.segmenters.base import segmenter_registry
67     from backend.pipeline.stitcher.compiler import VideoCompiler
68     from backend.pipeline.validators.base import registry
69     from backend.providers import provider_registry
70     from backend.reporting import emit_indexer_report
71     from backend.results import Failure # standardized error/result contract
72
73     # Server/network bootstrap helpers (CORS/host-allowlist config + the `indexer --app`
launcher)
74     # live in backend.server_runtime (P24 god-module split). Imported HERE, above `app =
FastAPI(...)`
75     # because the module-level add_middleware(...) below calls
_cors_origins_from_env/_allowed_hosts_from_env
76     # at import time; re-exported so `import backend.main as m; m._resolve_bind_host(...)`
keeps working.
77     from backend.server_runtime import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
78         _allowed_hosts_from_env,
79         _cors_origins_from_env,
80         _find_free_port,
81         _open_browser_when_ready,
82         _port_is_free,

```

```

83     _resolve_app_port,
84     _resolve_bind_host,
85 )
86 from backend.storage.capabilities import (
87     storage_capabilities, # P3: secret-free "which media can this box use" report
88 )
89 from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
90     ffmpeg_bin,
91     ffprobe_bin,
92 )
93 from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
94 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
95 from backend.utils.paths import (
96     output_base,
97     resolve_under_root,
98     safe_output_filename,
99     validate_input_path,
100 )
101 from backend.utils.redact import (
102     redact_secrets, # P0b: mask secret-shaped fields in /api/config
103 )
104 from backend.utils.single_instance import acquire_lock # P0e: one instance per database
105
106 app = FastAPI(title="Sports Highlight Indexer API")
107
108
109 def create_app(config: AppConfig, db: Database) -> FastAPI:
110     """Factory: inject shared state via ``app.state`` so route handlers access it
111 through
112     ``request.app.state`` ? eliminating the Optional module-level globals that forced
113     ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
114
115     Also sets the module-level ``global_config`` / ``db_instance`` for transitional
116     compatibility (helpers and tests still reference them). Route handlers use
117     ``request.app.state`` exclusively."""
118     global global_config, db_instance
119     global_config = config
120     db_instance = db
121     app.state.config = config
122     app.state.db = db
123     return app
124
125 # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
126 browsers per the
127 # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
128 credentials stay
129 # off. The origin list is configurable via the env var above (default [""]) so a hosted
130 deploy can
131 # lock it to its origin; the middleware is fixed at startup.
132 app.add_middleware(
133     CORSMiddleware,
134     allow_origins=_cors_origins_from_env(),
135     allow_credentials=False,
136     allow_methods=["*"],
137     allow_headers=["*"],
138 )
139
140 # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
141 honoured
142 # (a remote page can't read responses cross-origin AND its Host header is rejected
143 here). www_redirect
144 # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
145 app.add_middleware(

```

```

142     TrustedHostMiddleware,
143     allowed_hosts=_allowed_hosts_from_env(),
144     www_redirect=False,
145 )
146
147
148 # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
149
150
151 # Serves static frontend files directly (now under api/ per approved structure).
152 # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
an
153 # arbitrary directory finds the shipped assets instead of creating a stray
./backend/api/static.
154 # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
155 _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
156 try:
157     os.makedirs(_STATIC_DIR, exist_ok=True)
158 except OSError:
159     # The dir ships with the package (a no-op create); guard against a read-only install
160     # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
PermissionError.
161     pass
162 app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
163
164 # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
snapshot
165 # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
if it's
166 # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
no SPA change
167 # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
is enough
168 # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
169 _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
170 _UI_DIR = (
171     _BUNDLED_UI_DIR
172     if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
173     else _STATIC_DIR
174 )
175 _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
176 if os.path.isdir(_UI_ASSETS_DIR):
177     app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
178
179
180 @app.get("/", response_class=HTMLResponse)
181 def serve_index():
182     index_path = os.path.join(_UI_DIR, "index.html")
183     if os.path.exists(index_path):
184         with open(index_path, "r", encoding="utf-8") as f:
185             return f.read()
186     return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
187
188
189 def _bundled_ui_version() -> Optional[Dict[str, Any]]:
190     """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
191     Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
192     path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
193     try:
194         with open(path, "r", encoding="utf-8") as f:
195             return json.load(f)

```

```

196         except (OSError, ValueError):
197             return None
198
199
200     # Global variables initialized at startup
201     db_instance: Optional[Database] = None
202     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
203     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
204     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
205     _instance_lock: Optional[Any] = None
206
207     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
208     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
209     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
210     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
211     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
212     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
213     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
214     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
215         _MAX_DRAIN_WAKE_SEC,
216         JobWaiting,
217         _arm_wake_timer,
218         _drain_due_jobs,
219         _get_executor,
220         _job_to_request,
221         _maybe_record_job_cost,
222         _run_claimed_job,
223         _schedule_next_drain_if_waiting,
224         configure_drain,
225         resume_interrupted_remote_jobs,
226     )
227
228     # The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, Cloud Run
229     # dispatch+ingest, compile resolution + worker handler) now lives in
backend.orchestration
230     # (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressable
on
231     # backend.main ? `import backend.main as m; m._execute_processing(...)` , the job
worker's
232     # call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main,
...)
233     # all keep working unchanged.
234     from backend.orchestration import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
235         _cloud_available,
236         _CompileError,
237         _execute_compile,
238         _execute_processing,
239         _finalize_reingest,
240         _ingest_remote_segments,
241         _maybe_dispatch_remote,
242         _remote_input_size_bytes,
243         _report_config_for_input,
244         _resolve_compile,
245         read_path_rehydrate,
246         signed_reel_url,
247     )
248
249
250     # Legacy thin wrapper for any remaining direct calls during transition.

```

29. assistant (2026-07-06T17:11:19.840Z)

Let me read the orchestration module to see where detection is invoked:

30. user (2026-07-06T17:11:20.927Z)

```
1      """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3      The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4      Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5      and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6      Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8      SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9      monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import re
27     import tempfile
28     import threading
29     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
30     from urllib.parse import quote
31
32     from backend import storage
33     from backend.config import AppConfig, StitchingConfig
34     from backend.pipeline.segmenters.base import segmenter_registry
35     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
36     from backend.pipeline.validators.base import registry
37     from backend.results import Failure
38     from backend.utils.paths import output_base, safe_output_filename
39
40     if (
41         TYPE_CHECKING
42     ): # runtime import would be circular (main imports us for the re-export)
43         from backend.main import ProcessRequest # noqa: F401
44
45     logger = logging.getLogger(__name__)
46
47     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
```

```

compile stitch ?
48     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
49     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
50     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
51     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
61         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
62         """
63         if segments:
64             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
65         else:
66             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
67
68
69     def _validation_failure_message(failures: List[Any]) -> str:
70         details = []
71         for failure in failures:
72             name = getattr(failure, "validator_name", None) or "validation"
73             message = (getattr(failure, "message", None) or "").strip()
74             details.append(f"{name}: {message}" if message else str(name))
75         if not details:
76             return "Video failed validation checks."
77         return "Video failed validation: " + "; ".join(details)
78
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
scratch."""
82         storage_config = (
83             config.get("storage")
84             if isinstance(config, dict)
85             else getattr(config, "storage", None)
86         )
87         try:
88             if storage.input_is_local(raw_input, storage_config):
89                 return config
90         except ValueError:
91             return config
92         try:
93             cfg = (
94                 config.model_dump(mode="json")
95                 if hasattr(config, "model_dump")
96                 else dict(config or {})
97             )
98         except Exception:
99             cfg = {}
100         report = dict(cfg.get("report") or {})
101         if report.get("output_dir"):
102             return config
103         report["output_dir"] = os.path.join(output_base(config), "reports")
104         cfg["report"] = report

```

```

105         return cfg
106
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
112         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
113         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
114         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
115         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
116         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
117         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
118         and prunes any partial rows (destroy-AFTER-confirm)."""
119         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
120         ref = storage.parse_uri(uri)
121         n = 0
122         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123             local = storage.fetch(
124                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125             )
126             with open(local, newline="", encoding="utf-8") as f:
127                 for row in csv.DictReader(f):
128                     if n >= max_rows:
129                         logger.warning(
130                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
131                             max_rows,
132                             uri,
133                         )
134                         break
135                     try:
136                         start, end = float(row["start"]), float(row["end"])
137                     except (KeyError, TypeError, ValueError):
138                         continue
139                     if not (math.isfinite(start) and math.isfinite(end)):
140                         continue # reject inf/nan times rather than persist a junk rally
141                     meta: Dict[str, Any] = {
142                         "source": "cloud_run",
143                         "ending_reason": (row.get("ending_reason") or "").strip(),
144                     }
145                     sc = (row.get("shot_count") or "").strip()
146                     if sc:
147                         try:
148                             meta["shot_count"] = int(
149                                 float(sc)
150                             ) # OverflowError on 'inf'/'1e400'
151                         except (ValueError, OverflowError):
152                             meta["shot_count"] = sc
153                     db.add_play_segment(video_id, start, end, metadata=meta)
154                     n += 1
155         return n
156
157
158     def _cloud_available(cfg) -> bool:
159         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
160

```

```

161         True when the control plane is wired for Cloud Run: either the configured default
target is
162         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
163         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
164         are present so a forced per-request override can actually dispatch + collect a
result.
165
166         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work. """
167         deployment = getattr(cfg, "deployment", None)
168         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
169             return True
170         has_job = bool(
171             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
172         )
173         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
174         return bool(has_job and out_root)
175
176
177     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
178         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
179         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
180         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
181         (queue/cold-start), so the very first tick already shows movement. """
182         minutes = int(elapsed_sec) // 60
183         if minutes < 1:
184             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
185         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
186
187
188     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
189         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
190         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
191         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
192         ``md5Hash``. Never raises ? a metadata miss must not block a submit. """
193         if getattr(input_ref, "backend", "") != "gcs":
194             return None
195         sc = _storage_settings(cfg)
196         if not sc:
197             return None
198         try:
199             st = storage.get_storage("gcs", config=sc).stat(input_ref)
200             return getattr(st, "md5", None) or None
201         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
202             logger.info(
203                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
204                 getattr(input_ref, "uri", input_ref),
205                 exc_info=True,
206             )
207         return None
208
209
210     def _cached_process_payload(
211         video_id: str, validation_results, segments, requested_compute
212     ) -> Dict[str, Any]:
213         """The /api/process result for a video whose detection already exists ? ONE shape
for
214         both producers (the drain's DB-cache short-circuit and the submit-time cache,

```



```

#484)."""
215         return {
216             "video_id": video_id,
217             "status": "success",
218             "message": (
219                 f"Reused {len(segments)} cached play segments. "
220                 "Pass force=true to reprocess this video."
221             ),
222             "results": validation_results or [],
223             "play_segments": segments,
224             "cached": True,
225             "compute": {"path": "cached", "requested": requested_compute},
226         }
227
228
229     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
230         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
232         decide."""
233         ref = storage.parse_uri(uri)
234         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
235             local = storage.fetch(
236                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
237             )
238             with open(local, encoding="utf-8") as f:
239                 loaded = json.load(f)
240             return loaded if isinstance(loaded, dict) else None
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
243     hits
244     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
245     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
246     instance
247     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
248     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
249     _REHYDRATE_LOCKS_GUARD = threading.Lock()
250
251     def _rehydrate_lock(video_id: str) -> threading.Lock:
252         with _REHYDRATE_LOCKS_GUARD:
253             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
254
255     def probe_process_cache(db, cfg, video_id: str) -> bool:
256         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
257         bucket)?
258         1
259         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
260         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
261         (blocker 2): a True probe means the request should never need CP scratch. Best-
262         effort:
263         errors report False (? the normal dispatch path with its guards)."""
264         try:
265             existing = db.get_video(video_id)
266             if (
267                 existing
268                 and existing.get("status") == "processed"
269                 and db.query_play_segments(video_id, {})
270             ):
271                 return True
272             sc = _storage_settings(cfg)
273             if not sc or (sc.get("backend") or "local") == "local":
274                 return False

```

```

273         output_root = (sc.get("output_root") or "").rstrip("/")
274         if not output_root:
275             return False
276         report_uri = f"{output_root}/outputs/{video_id}/report.json"
277         if not storage.exists(report_uri, config=sc):
278             return False
279         report = _read_bucket_json(report_uri, sc)
280         return bool(report and report.get("status") == "success")
281     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282         logger.warning(
283             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284         )
285         return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (`outputs/<md5>/report.json` with `status=success`
293         re-ingests via `_ingest_remote_segments`): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). `None` ?
no
295         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
296         branch mirrors the real cloud ingest exactly (watermarks + `_finalize_reingest`),
so a
297         partial earlier ingest is replaced, never double-counted.
298
299         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
300         per-`video_id` lock ? a concurrent caller blocks, then finds the winner's rows via
the

```

31. user (2026-07-06T17:11:21.667Z)

```

795:         result = segmenter.process_video(

```

32. assistant (2026-07-06T17:11:24.255Z)

Let me continue reading the orchestration to find where the actual segmentation happens:

33. user (2026-07-06T17:11:24.917Z)

```

750         logger.info("[API] Video passed checks. Segmenting and indexing...")
751         db.add_video(
752             video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
753         )
754
755         segmenter_cls = segmenter_registry.get(seg_name)
756         if not segmenter_cls:
757             # Submit-time validation already 400s unknown names; this is the defensive
758             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
759             available = list(segmenter_registry.list_available().keys())
760             response = {
761                 "video_id": video_id,
762                 "status": "failed",
763                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
764                 "results": [r.model_dump() for r in val_results],
765                 "play_segments": [],
766             }

```

```

767         return response
768
769         logger.info(f"[API] Using segmenter: {seg_name}")
770         notify(f"segmenting ({seg_name})")
771         segmenter_actual = seg_name
772         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
773         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
774         play_wm, cand_wm = db.segment_watermarks(video_id)
775         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
776         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
777         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
778         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
779         # they're dropped on the cloud path in this slice ? a documented follow-up.)
780         remote = _maybe_dispatch_remote(
781             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
782         )
783         if remote is not None:
784             response, segmentation_ran = (
785                 remote # `ran` = did the cloud job actually execute (report)
786             )
787             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
788         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
789         compute_actual = "in_process"
790         segmenter = segmenter_cls(db, config)
791         try:
792             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
793             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
794             with _LOCAL_COMPUTE_LANE:
795                 result = segmenter.process_video(
796                     video_id, local_video, req.player_pool, req.max_frames
797                 )
798         except Exception:
799             # A raising segmenter must not leave half-written rows: restore the pre-run
800             # state (same destroy-after-confirm contract as the Failure branch), then
let
801             # the job worker record the failure.
802             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
803             raise
804         segmentation_ran = True
805
806         if isinstance(result, Failure):
807             logger.error(f"[API] Segmenter failed: {result.message}")
808             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
809             response = {
810                 "video_id": video_id,
811                 "status": "failed",
812                 "message": f"Segmenter '{seg_name}' failed: {result.message}",
813                 "results": [r.model_dump() for r in val_results],
814                 "play_segments": [],
815             }
816             return response
817
818         segments = result.value if hasattr(result, "value") else result
819         notify("finalizing")
820         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

```

```

821
822         response = {
823             "video_id": video_id,
824             "status": "success",
825             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
.value
826             "results": [r.model_dump() for r in val_results],
827             "play_segments": segments,
828         }
829         return response
830     finally:
831         # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
832         # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
833         # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
834         # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
835         if isinstance(response, dict):
836             response.setdefault(
837                 "compute",
838                 {
839                     "path": compute_actual or "not_run",
840                     "requested": getattr(req, "compute", None),
841                 },
842             )
843         prov = response.get("compute", {}) if isinstance(response, dict) else {}
844         logger.info(
845             "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
846             video_id,
847             getattr(req, "compute", None),
848             prov.get("path"),
849             prov.get("dispatched"),
850             (response or {}).get("status") if isinstance(response, dict) else None,
851         )
852         # Always emit the per-video observability report (next to the video, or to
853         # report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
854         # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
855         # doesn't surface arbitrary run signals.)
856         paths = _main.emit_indexer_report(
857             db=db,
858             video_id=video_id,
859             # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
860             # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
861             # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
862             # still preserved in DB provenance via db.add_video.
863             video_path=local_video,
864             config=_report_config_for_input(config, req.video_path),
865             val_results=val_results,
866             val_context=val_context,
867             segmenter_requested=req.segmenter,
868             segmenter_actual=segmenter_actual,
869             was_reingest=was_reingest,
870             segmentation_ran=segmentation_ran,
871         )
872         if paths and isinstance(response, dict):
873             response["reports"] = paths
874         storage.cleanup_acquired_local_input(

```

```

875         req.video_path, local_video, getattr(config, "storage", None)
876     )
877
878
879     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
880         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
881
882         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
883         that needs a
884         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
885         guard degrades
886         to a no-op rather than blocking a job it cannot measure."""
887         try:
888             sc = (
889                 storage_config.model_dump()
890                 if hasattr(storage_config, "model_dump")
891                 else (storage_config or {})
892             )
893             backend = storage.get_storage(input_ref.backend, config=sc)
894             return int(backend.stat(input_ref).size)
895         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
896             not block
897             logger.debug(
898                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
899             )
900             return None
901
902     class _CompileError(Exception):
903         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
904         the API path
905         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
906         result."""
907
908         def __init__(self, status_code: int, detail: str):
909             super().__init__(detail)
910             self.status_code = status_code
911             self.detail = detail
912
913     def _resolve_compile(
914         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
915     ):
916         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
917         resolve, since a
918         row could change between submit and run). Verifies the video exists, the requested
919         segments match
920         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
921         shot_count are exempt
922         so manual/legacy picks can't silently vanish), and the output name is traversal-
923         safe. Raises
924         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
925         safe_out_name)``."""
926         video = db.get_video(video_id)
927         if not video:
928             raise _CompileError(404, "Indexed video record not found.")
929         # Reject path traversal / absolute names (os.path.join would otherwise write
930         anywhere on disk).
931         try:
932             out_name = safe_output_filename(output_filename)
933         except ValueError as e:
934             raise _CompileError(400, str(e)) from e
935
936         all_segments = db.query_play_segments(video_id, {})
937         matched = [s for s in all_segments if s["id"] in segment_ids]

```

```

929         if not matched:
930             raise _CompileError(400, "No matching play segments found to stitch.")
931
932     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
933     kept = []
934     for s in matched:
935         meta = s.get("metadata") or {}
936         sc = meta.get("shot_count") if isinstance(meta, dict) else None
937         try:
938             if sc is not None and int(sc) < stitching.min_shot_count:
939                 continue
940         except (TypeError, ValueError):
941             pass # unparseable count -> treat as unknown, keep
942         kept.append(s)
943     if len(kept) < len(matched):
944         logger.info(
945             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
946             stitching.min_shot_count,
947             len(matched) - len(kept),
948             len(matched),
949         )
950     if not kept:
951         raise _CompileError(
952             400,
953             f"All {len(matched)} matching segments fall below "
954             f"stitching.min_shot_count={stitching.min_shot_count}.",
955         )
956     return video, kept, out_name
957
958
959     def reel_object_uri(cfg, video_id: str, name: str) -> str:
960         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
(#463), or "" when
961         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
962         mirror-upload and the /api/highlights redirect:
`<output_root>/highlights/<video_id>/<name>`. """
963         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
964         if not root or "://" not in root:
965             return ""
966         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
967
968
969     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
970         """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
971         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
972         isn't silently constructed as a fresh default. """
973         sc = getattr(cfg, "storage", None)
974         return sc.model_dump() if sc is not None else None
975
976
977     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
978         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
979         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
980         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
981         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24). """
982         reel_uri = reel_object_uri(cfg, video_id, name)
983         if not reel_uri:

```

```

984         return None
985     settings = _storage_settings(cfg)
986     if storage.exists(reel_uri, config=settings):
987         return storage.signed_url(reel_uri, config=settings, method="GET")
988     return None
989
990
991     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
992         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
993         stitch the reel,
994         and return the {status, filepath, download_url} result the SPA polls for. Returns a
995         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
996         stitch error)
997         so the SPA always gets a clean verdict instead of a worker-crash error."""
998         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
999         backend.main
1000
1001         notify = progress or (lambda stage: None)
1002         params = json.loads(job.get("params") or "{}")
1003         video_id = job.get("video_id") or ""
1004         segment_ids = params.get("segment_ids") or []
1005         locale = params.get("locale")
1006
1007         notify("resolving")
1008         try:
1009             video, kept, out_name = _resolve_compile(
1010                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1011             )
1012         except _CompileError as e:
1013             return {"status": "failed", "message": e.detail, "video_id": video_id}
1014
1015         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1016         output_dir = (
1017             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1018         )
1019         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
1020         the job);
1021         # unknown/absent locale keeps the configured default unchanged.
1022         localized_watermark = localize_watermark(stitching.watermark, locale)
1023         compiler = _main.VideoCompiler(
1024             output_dir,
1025             begin_padding=stitching.begin_padding,
1026             end_padding=stitching.end_padding,
1027             watermark=localized_watermark,
1028         )
1029         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1030
1031         notify("stitching")
1032         local_src = ""
1033         try:
1034             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
1035             path). Resolve to
1036             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
1037             scratch for a URI (the
1038             # same seam the process path uses); without it, compiling a bucket-ingested
1039             video can't open gs://.
1040             local_src = storage.acquire_local_input(
1041                 video["filepath"], getattr(cfg, "storage", None)
1042             )
1043         except:
1044             # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
1045             (for remote
1046             # dispatch) doesn't run multiple compiles at once on the box (see
1047             _LOCAL_COMPUTE_LANE).
1048             with _LOCAL_COMPUTE_LANE:
1049                 out_path = compiler.compile_highlights(local_src, time_pairs, out_name)

```

```

1040         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1041         logger.error(
1042             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1043         )
1044         return {
1045             "status": "failed",
1046             "message": f"Highlight stitching failed: {e}",
1047             "video_id": video_id,
1048         }
1049     finally:

```

34. assistant (2026-07-06T17:11:27.027Z)

Perfect. Now let me search for where the timestamps are actually used during compilation and how they relate to the original video:

35. user (2026-07-06T17:11:28.430Z)

```

240:         per-segment body of compile_highlights). Returns (clip_path, skip_record): on
254:             f"{segment_label}: start_time is negative ({start:.2f}s). Clamping to 0."
260:             f"{segment_label}: start_time ({start:.2f}s) >= end_time ({end:.2f}s).
Skipping invalid segment."
268:             f"{segment_label}: start_time ({start:.2f}s) is beyond the video
duration ({video_duration:.2f}s). "
277:             f"{segment_label}: end_time ({original_end:.2f}s) exceeds video duration
({video_duration:.2f}s). "
278:             f"Clamping end_time to {video_duration:.2f}s. The segment may be
truncated."
382:     def compile_highlights(

```

36. assistant (2026-07-06T17:11:29.697Z)

Let me read the compile_highlights method:

37. user (2026-07-06T17:11:30.104Z)

```

380         return None, (idx, start, end, "empty output clip")
381
382     def compile_highlights(
383         self,
384         raw_video_path: str,
385         segments: List[Tuple[float, float]],
386         output_filename: str,
387     ) -> str:
388         """
389         Extracts selected segments from a raw video and stitches them together
390         into a seamless highlights file, applying end_padding to prevent abrupt endings.
391
392         .. warning::
393             This method uses a two-stage codec contract: per-clip re-encodes must use
394             IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
395             concat can stream-copy (`-c copy`) without re-encoding. Changing the
396             clip encode settings without updating the concat assumptions will produce
397             broken output.
398         """
399         if not segments:
400             raise ValueError("No highlight segments provided to compile.")
401
402         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
403         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
404         original_count = len(segments)
405         segments = self._merge_overlapping(segments)
406         if len(segments) < original_count:

```



```

407         logger.info(
408             "Merged %d overlapping/duplicate segment(s) before stitching: %d ->
%d.",
409             original_count - len(segments),
410             original_count,
411             len(segments),
412         )
413     if not segments:
414         raise ValueError("No valid (start < end) highlight segments after merge.")
415
416     if not os.path.exists(raw_video_path):
417         raise FileNotFoundError(
418             f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
419         )
420
421     # Defense-in-depth: never let the output name escape output_dir (the API also
422     # validates this at the request boundary).
423     output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
424
425     # Probe video duration so we can detect out-of-range segments
426     video_duration = self._get_video_duration(raw_video_path)
427     if video_duration > 0:
428         logger.info(f"Source video duration: {video_duration:.2f}s")
429     else:
430         logger.warning(
431             "Video duration unknown. Cannot validate segment boundaries against
video length."
432         )
433
434     # We will create temporary clips and stitch them using FFmpeg concat demuxer
435     temp_clips = []
436     skipped_segments = []
437
438     # Create a temp directory within output_dir
439     with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
440         logger.info(
441             f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
442         )
443         # Build the (optional) watermark filtergraph once; it is applied during
every
444         # per-clip re-encode so the mark is baked into each clip identically.
445         watermark_filter = self._watermark_filter(tmp_dir)
446         if watermark_filter:
447             logger.info("[watermark] applying branding overlay to each clip.")
448         for idx, (raw_start, raw_end) in enumerate(segments):
449             clip_path, skip_record = self._trim_one_segment(
450                 idx,
451                 raw_start,
452                 raw_end,
453                 segments,
454                 video_duration,
455                 tmp_dir,
456                 raw_video_path,
457                 watermark_filter,
458             )
459             if clip_path is not None:
460                 temp_clips.append(clip_path)
461             if skip_record is not None:
462                 skipped_segments.append(skip_record)
463
464         # Report summary of skipped segments
465         if skipped_segments:
466             logger.info(
467                 f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}

```

```

segments were SKIPPED:"
468         )
469         for seg_idx, seg_start, seg_end, reason in skipped_segments:
470             logger.info(
471                 f" - Segment #{seg_idx + 1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}"
472             )
473             logger.info("")
474
475         if not temp_clips:
476             raise RuntimeError(
477                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
478                 f"All {len(segments)} segments were skipped. "
479                 f"Check the errors above for details. "
480                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
481             )
482
483         logger.info(
484             f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch..."
485         )
486
487         # Create a list file for FFmpeg concat demuxer
488         list_file_path = os.path.join(tmp_dir, "clips.txt")
489         # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
490         # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
491         with open(list_file_path, "w", encoding="utf-8") as f:
492             for clip in temp_clips:
493                 # The concat demuxer resolves relative entries against the list
file's
494                 # directory. Use absolute paths so a temp-dir clip does not become
495                 # output/tmp.../output/tmp.../clip_0.mp4 on Windows.
496                 safe_path = Path(clip).resolve().as_posix().replace("'", "'\\'")
497                 f.write(f"file '{safe_path}'\n")
498
499         logger.info("Stitching clips together...")
500         # Stitch them together without re-encoding (since they were encoded with
identical settings)
501         concat_cmd = [
502             ffmpeg_bin(),
503             "-y",
504             "-f",
505             "concat",
506             "-safe",
507             "0",
508             "-i",
509             list_file_path,
510             "-c",
511             "copy",
512             output_path,
513         ]
514
515         try:
516             timeout = ffmpeg_timeout_sec()
517             subprocess.run(
518                 concat_cmd, capture_output=True, check=True, timeout=timeout
519             )
520
521             # Final verification
522             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
523                 raise RuntimeError(
524                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
525                     f"but output file is missing or empty: {output_path}"

```

```

526         )
527
528         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
529         logger.info(
530             f"Dynamic highlight reel saved: {output_path} ({output_size_mb:.1f}
MB)"
531         )
532         return output_path
533     except subprocess.CalledProcessError as e:
534         stderr_msg = e.stderr.decode(errors="replace").strip()
535         raise RuntimeError(
536             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
537             f"  Command: {' '.join(concat_cmd)}\n"
538             f"  FFmpeg stderr: {stderr_msg[-500:]}"
539         ) from e
540     except subprocess.TimeoutExpired as e:
541         raise RuntimeError(
542             f"[COMPILER ERROR] FFmpeg concatenation timed out after
{timeout_label(timeout)}.\n"
543             f"  Command: {' '.join(concat_cmd)}"
544         ) from e
545

```

38. assistant (2026-07-06T17:11:31.803Z)

Perfect. Now let me check the GitHub issue #483 reference to understand what's being proposed:

39. user (2026-07-06T17:11:33.563Z)

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md:712:- **Cost/new surface:** a new `deployment`+`compute\_target` config surface, a Cloud Run **dispatch shim**, a **containerized GPU+ffmpeg worker image**, a **cost-ledger migration**, and **edge auth** must all be built + maintained. Cross-GPU detection is **not** byte-identical ? parity asserted at the window level. The `byo\_worker` tier carries a real ops/support burden (deferred, power-user only).

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md:482:- reasoning: Single-worker executor: backend/api/job_worker.py:47-49 (max_workers=1) with the in-code "O2 RISK" admission at :32-39 that one hung job blocks all subsequent jobs and "the executor itself cannot enforce timeouts on the Thread". Serving-path ffmpeg/ffprobe calls with NO timeout=: (compile job, POST /api/compile main.py:1594 -> _execute_compile -> compiler.compile_highlights at main.py:1686, no wrapper) backend/pipeline/stitcher/compiler.py:130 (ffprobe), :321 (per-clip libx264 encode), :494 (concat); (process job, _execute_processing main.py:1132) validators backend/pipeline/validators/format.py:43, pts_continuity.py:35, resolution_fps.py:15; backend/utls/video.py:18,:75 (chunking); backend/pipeline/segmenters/gemini.py:166; backend/pipeline/audio/hit_detector.py:53. The mitigations listed in job_worker.py:34-37 do NOT cover ffmpeg: run_bounded wraps only the Gemini generate call (backend/providers/gemini.py:233, op_timeout_sec=180); detector timeout_sec covers only GPU runner subprocesses (detectors/base.py:116, native_wasb_runner.py:134). Recovery is restart-only: fail_stale_jobs is called once at startup (main.py:2030; database.py:1122-1125) and marks ALL queued/running jobs failed. No fix landed: git head e410136; commit 24178a7 (#374) documents the O2 risk but adds no timeouts. Secondary confirmed defect: /api/source proxy warm (main.py:657 daemon thread) runs annotation_proxy.py:81 ffmpeg with no timeout; a hang leaks the _proxy_inflight dedup entry forever (main.py:607-620), permanently disabling proxy generation for that video (does not wedge the job queue).

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/archives/past_projects/AUDIT_REPORT_khelsutra-guru_2026-07-02-COMPLETED-2026-07-02.md:483:- notes: Claim is literally true, but "permanently wedges" should be "wedges until server restart, and the restart sweep (fail_stale_jobs) then fails every queued job". Sharper failure scenario: POST /api/compile for a video whose stored local filepath sits on a network-backed mount (Google Drive streaming / SMB) ? storage.acquire_local_input is identity for local paths (main.py:1683) ? the mount stalls mid-read (or the MP4 is truncated/corrupt and ffmpeg's demuxer loops); subprocess.run at compiler.py:321 blocks forever; the compile job stays 'running'; every subsequent /api/process

and /api/compile returns 202 but stays 'queued' indefinitely with zero error surfaced (drains submitted at main.py:1635 queue behind the blocked task on the max_workers=1 executor); only a full restart recovers, at the cost of failing the whole queue. Severity P2 not P0/P1: the trigger class is rare (mitigations already bound the historically hang-prone ops ? Gemini generate 180s, GPU/WSL detector timeout_sec 1800s, Cloud Run poll max_wait_sec), the risk is explicitly documented in-code, and restart recovers; but impact when triggered is a silent total outage of the job pipeline. Fix (S, mechanical): add timeout= to the ~10 serving-path ffmpeg/ffprobe subprocess.run/check_output sites (config-driven, e.g. 30-60s for ffprobe probes, stitching-level 1800s default for encode/concat/chunking) and map subprocess.TimeoutExpired to the existing clean-job-failure paths (compiler already converts CalledProcessError to a failed-job result at main.py:1687-1695; annotation_proxy already catches broadly at annotation_proxy.py:82). The broader per-job wall-clock timeout job_worker.py:38 calls a "future slice" is a separate M item and not required to close this finding. Not OWNER_GATED ? timeout defaults are an engineering choice with no spend/product implications.

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md:35: for proxy transcode + batch transcode ? defaults to CPU `libx264`. Shares the same local-filesystem + MD5

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md:67:2.

****Store byte-identical files**** the labels were made against. If you must proxy-transcode for size, carry

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/06-RISKS-AND-OPEN-QUESTIONS.md:35: proxy transcode changes it, and the export `manifest.json` ****omits `md5`.** ****Mitigation:**** ****add `md5` to**

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/BACKLOG.md:13:- ? ****One-shot "serving startup-script" for the GPU VM**** ? `create\_gpu\_vm.sh` already injects a metadata `startup-script` (`deploy/gcp/ops-agent-startup.sh`), but it only installs the ****Ops Agent****; standing up the live engine on the VM (`khelsutra/deploy/GPU\_VM\_SERVING.md`) is still manual. Propose a `deploy/gcp/serving-startup.sh` fed via `--metadata-from-file=startup-script=?` (or pasted into the GCP console) that, on boot: installs ffmpeg/cv2 libs ? pulls the engine source (from GCS) + builds the venv ? ***(optional)*** the WASB native env (`gpu\_setup.sh` + weights) ? writes the serving `config.json`/`engine.env` ? installs `cloudflared` with the tunnel token ? starts `khelsutra-engine`. So a GPU serving box comes up ****hands-off**** (reuses the `deploy/gcp/nightly\_regression/run\_on\_vm.sh` bootstrap pattern). ****Why deferred ?** the design hinge is **SECRETS:** ****** the Gemini key / CF-Access AUD / tunnel token must ****not**** sit in a plaintext startup-script (instance metadata is broadly readable by anyone with `instances.get`); they should come from ****Secret Manager**** (VM SA ? `secretAccessor`) or be injected post-boot. Needs that small secrets-from-metadata design + which config is baked vs fetched ? not just a script (owner asked 2026-06-23). ****Revisit:**** when the GPU-VM serving path is used often enough to automate.

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/CODE_MAP.md:138:- `@scripts/doctor.py` (formerly root `setup.py`) ? `::init\_default\_config` ? delegates to `backend.config.save\_default\_config` (single-source defaults; skips if config.json exists). `::run\_pip\_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run\_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA). Run via `python scripts/doctor.py` or `indexer --setup`.

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/CODE_MAP.md:297:- ****ffmpeg NOT in WSL**** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract\_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames\_dir`. ? Don't import `wasb\_infer` from Windows code (it imports torch + WASB repo modules).

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/CODE_MAP.md:348:All pure-logic / mocked-boundary unit tests; no WSL/GPU/Gemini/ffmpeg actually invoked. `@run\_all\_tests.py` (repo root) is the suite runner.

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/README.md:121:| M6 | Cloud Run GPU ****dispatch shim**** (control plane ? job via storage ? re-claim) + ****containerized worker image**** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287) (shim+image+mocked ?; real build/run = M7 ? owner) |

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/docs/COMPUTE_DECOUPLED_SERVING/TESTING_STRATEGY.md:38:- ****How:**** owner runs the ****golden corpus**** on the GPU box and on Cloud Run; compare rally counts/windows; ****smoke-test the reel**** (`ffmpeg -f null -` full decode exit 0 + a 5-frame contact sheet visually confirming real

rallies). Recorded in a ****DEPLOY_REPORT**** under `runlogs/`.

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:14:> #471 #480 #482 #483 #484 #485 (plus the already-shipped #479-config, #481, and khehsutra

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:66:**Read first:** [[alpha-cuj-testing-2026-07-05]] · issues #480/#471/#484/#485/#482/#483 ·

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:83:| D10 | Any quality-affecting flip (#483 proxy) ships a ****Launch Report**** gated on the golden set | standing convention [[launch-report-convention]] | honest measured results only; most-conservative-passing setting wins |

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:89:- ****Downsample experiment (validates #483):**** dev RTX 2070 Super GPU-transcoded the full 5.9-min

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:176:- Worker `--task-timeout 3600` stays for now; #483 (below) is the real fix for long footage, not a

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:186:### 4.5 Cost ? downsample-for-detection (P7 · #483 · golden-gated)

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:239:| Proxy detection regresses quality (#483) | worse rallies for testers | D10 golden gate + Launch Report before default-ON; most-conservative-passing setting |

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:251:- The golden set (n=15) can only prove #483 on footage classes it contains; new footage classes get

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:274:| P7 | #483 downsample-for-detection behind config; default set by the golden gate | engine | P0 | ****Golden gate + Launch Report (D10)**** | ? | ? |

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:287:3. #483 bundling ? ? ****P9 does NOT wait**** for the golden gate; #483 follows in the next image.

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:337:**Issues:** engine #471 #479 #480 #481(merged `0a7b3ff`) #482 #483 #484 #485; khehsutra #153 #154

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:347: answered on the recommendations (8 GiB/2 vCPU · P9 doesn't wait for #483 · 60-min interim

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/CONTROL_PLANE_DURABLE_REBUILD.md:369: P9 deploy). P7/#483 remains golden-gated and does not hold P9 (\$8 Q3).

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/DECISION_SNAPSHOT_REGRESSION.md:17:A deterministic regression that proves a change is ****quality-neutral**** (behavior-preserving) by freezing and comparing the pipeline's ****decision output**** ? the rally segment list ****+ the stitch/concat plan**** ? run off a ****cached trajectory**** (GPU-free). It ****extends**** the existing characterization snapshot

(`test\_replay\_matches\_committed\_expected`, today segmentation-only) down to the stitch plan, runs offline in CI, and gives the ****behavioral proof**** that lets a **genuine** refactor (not just `ruff format`) be confidently called quality-neutral. ****Most important caveat: never hash the encoded reel `.mp4`**** ? ffmpeg/x264/NVENC/mux-timestamps make those bytes non-deterministic, so a reel-MD5 would false-alarm on every ffmpeg bump.

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/EXECUTION_POLICY.md:17:The same shape recurs anywhere we wait on an external/long thing (WSL/WASB GPU pass, ffmpeg, a future

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/EXECUTION_POLICY.md:103:- ****P4 ? apply the same policy to the WSL/WASB GPU pass + ffmpeg**** (the other long ops).

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md:87:- ****Platform = no Python hot path to escape**** ? the heavy bytes/GPU already run out-of-process (ffmpeg, WASB-in-WSL, cloud

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer/docs/PACKAGING_PLAN.md:12:****You are here (2026-06-22):**** ****v1 LIGHT is FEATURE-COMPLETE.**** `pip`/`uv` tool install ? `indexer --app` boots torch-free, seeds a truly-local

`**motion** default (gate 08), and **serves the 3-step first-run wizard** single-origin (P2c, bundled), with app-home state / env-overridable ffmpeg / DNS-rebinding-guarded bind / single-instance lock / safe DB upgrades / clean install+uninstall. **P3 GPU-validated on a real L4 ($0) ? 22 rallies ? 73.7 MB reel.** Remaining = OWNER clean-box acceptance + ffmpeg-acquisition UX; then the north-star (P3-cloud/P4). Ramp up via $7 + memory `[[packaging-app-plan]]`/`[[next-session-packaging-app]]`.`

40. assistant (2026-07-06T17:11:35.136Z)

Excellent! I found it. Now let me search for more specifics about issue #483:

41. user (2026-07-06T17:11:36.370Z)

3. Foundation ? measured evidence this plan rests on

- `**Failure chain + gs:// pass** ? `[verified]` live gcloud logs + owner screenshots, 2026-07-05 ($1; memory [[alpha-cuj-testing-2026-07-05]]).`
- `**Downsample experiment (validates #483):** dev RTX 2070 Super GPU-transcoded the full 5.9-min 1080p60 clip to 1080p30 in **36 s (~10x realtime)** via NVDEC?`scale_cuda`?NVENC vs ~3.5 min CPU libx264 `[verified]`. NVDEC/NVENC are dedicated hardware blocks ? they don't contend with CUDA inference. Expected effect: detection ~22 min ? ~5 min at a 30 fps proxy.`
- `**WASB throughput:** ~25 fps on the L4 (a 6-min 1080p60 clip ? 21 600 frames ? 22 min) `[verified 2026-07-05]`.`
- `Platform facts `[researched ? re-verify during P9]`: Cloud Run service volumes are in-memory emptyDir / GCS FUSE / NFS only (no persistent local disk); memory up to 8 GiB runs on 2 vCPU, above 8 GiB requires 4; GCS non-composite objects expose `md5Hash` metadata; GCS resumable upload sessions remain valid across minutes-long gaps between chunks (they are designed for exactly this).`

--

`WASB runner emits periodic progress (frames done / % / rough ETA) ? `set_job_progress` ? `/api/jobs/{id}.progress` returns e.g. `"segmenting 42% (~6 min left)"` ? the SPA shows it (and it feeds the 4.3 stall detector). A per-job runlog artifact (phases, timings, counts) is written alongside `outputs/<md5>/` for after-the-fact triage. `[design ? issue #482 carries the full ask]``

4.5 Cost ? downsample-for-detection (P7 · #483 · golden-gated)

Detection does not need 1080p60; the reel does. Worker-side: when estimated processing time crosses a threshold (from `res.fps.duration` vs the known ~25 fps L4 throughput), GPU-transcode a proxy (candidate: 720p30 ? the golden gate picks the most aggressive setting that holds quality), run WASB on the **proxy**, map rally timestamps back, cut the reel from the **ORIGINAL**. User-visible copy explains it. **Gate (D10):** golden-set (n=15) F1/tolF1 at the proxy settings vs baseline; ship default-ON only on a no-material-regression result, with a Launch Report. Payoff: ~22 min ? ~5 min on the 2026-07-05 clip class, GPU cost cut proportionally, and the 4.3 timeout pressure mostly evaporates. `[design; experiment evidence \$3]`

--

`| P3 | Restart-safe remote jobs: md5-keyed startup recovery (extend `recover_stale_jobs`) + read-path re-hydrate via `_ingest_remote_segments` (#471 #485) | engine | P4 | No | ? | [#494](https://github.com/Khelsutra/badminton-highlight-indexer/pull/494) |`

`| P4 | Submit-time MD5 on the job row + completed-work cache-reuse with `force` bypass (#484) | engine | P2 (upload hash) | No | ? | [#492](https://github.com/Khelsutra/badminton-highlight-indexer/pull/492) |`

`| P5a | `deployment.remote_poll_max_wait_sec` knob (default 3900) + emit `max_upload_mb`/`max_part_mb` in `/api/capabilities` (#480 item 5 ? revives the SPA pre-flight)`

| engine | P0 | No | ? | [#491](https://github.com/Khelsutra/badminton-highlight-indexer/pull/491) |
| P5b | SPA: both poll ceilings 10 ? 60 min (pre-flight revived server-side by P5a ? no SPA change needed) | khelsutra | P5a | No | ? |
[khelsutra#162](https://github.com/avidullu/khelsutra/pull/162) |
| P6 | #482 progress heartbeat ? `job.progress` (CP-side slice shipped; worker-side frame-% + per-job runlog stay on #482) | engine (+khelsutra) | P0 | No | ? |
[#493](https://github.com/Khelsutra/badminton-highlight-indexer/pull/493) |
P7	#483 downsample-for-detection behind config; default set by the golden gate	engine	P0	**Golden gate + Launch Report (D10)**	?	?
P8	Re-vendor the SPA snapshot (`scripts/bundle\_ui.sh`) at khelsutra master ? P5b	engine	P5b	No	?	[#495](https://github.com/Khelsutra/badminton-highlight-indexer/pull/495) (snapshot @ khelsutra `90d1d90`)
P9	**REBUILD + REDEPLOY + browser-CUJ verify + runlog** (\$4.8, \$9)	ops	P1?P5b, P8 (P6/P7 if ready)	**Owner: spec confirm + deploy/spend authorization**	?	executed 2026-07-05: build `47877373` ? digest `3bfefaf6?` ? rev **00012-wxk** (8Gi, pins, auth ON); [runlog](COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_cp-rebuild_2026-07-05.md)

****Bundling note:**** P6/P7 ride the same image if ready before P9 executes; otherwise they trigger a cheap follow-up rebuild. ****P9 must not wait on them**** ? it is the DoD #1 blocker.

8. Open questions ? owner ***(ANSWERED ? owner LGTM 2026-07-05, recommendations stand)***

1. ****Confirm this spec + authorize P9**** ? ? ****CONFIRMED**** (in-session owner LGTM on this ask + review LGTM on [#487](https://github.com/Khelsutra/badminton-highlight-indexer/pull/487)).
--
- 2026-07-05 (night) ? execution sweep under the owner's review-gate (verified-LGTM per PR):
P1 #489 · P2 #490 · P4 #492 · P5a #491 · P6 CP-slice #493 · P5b khelsutra#162 all MERGED;
P3 #494 in review (review hardened it: forced rows + non-cloud rows never resume from the cache; compute normalization matches execution); P8 re-vendors the SPA at khelsutra `90d1d90` (#153/#155/#158 + the 60-min ceilings ? the live medium-picker trap dies with the P9 deploy). P7/#483 remains golden-gated and does not hold P9 (\$8 Q3).

42. assistant (2026-07-06T17:11:38.279Z)

Perfect! Now I have all the information I need. Let me create a comprehensive findings report:

43. user (2026-07-06T17:11:38.721Z)

Chapter marked: "Findings Report". Continue your current work.

44. assistant (2026-07-06T17:12:15.560Z)

FINDINGS REPORT: Resolution/FPS Flow & GPU-ffmpeg Proxy for WASB Detection (Issue #483)

1. ****PROXY/TRANSCODE STATUS: Not Yet Implemented****

****Confirmation:**** Issue #483 is ****proposed but NOT shipped****. The design and experiment are documented; implementation is deferred to P7 (Post-alpha) behind a golden-gate quality validation. The CONTROL_PLANE_DURABLE_REBUILD.md roadmap lists it as "? (incomplete)" with dependency on Launch Report (D10).

****Evidence:****

- `docs/CONTROL\_PLANE\_DURABLE\_REBUILD.md:186-239` describes P7 #483 downsample-for-detection as golden-gated, awaiting measured quality validation.
- `docs/CONTROL\_PLANE\_DURABLE\_REBUILD.md:89-92`: Experiment verified 1080p60 ? 1080p30 transcode in **36s via NVDEC?scale_cuda?NVENC** (10x faster than CPU libx264's 3.5min); no default setting yet.
- No transcode/proxy call exists in `backend/orchestration.py` line 795 where

`segmenter.process\_video(video\_id, local\_video, ...)` is invoked.

2. **CONFIG FIELDS CONTROLLING RESOLUTION/FPS FOR DETECTION**

****In `config.json` (validation only, NOT detection input)**:**

```
```json
"validation": {
 "min_resolution_width": 1280, // line 6 ? rejects videos below this
 "min_resolution_height": 720, // line 7
 "min_fps": 29.9 // line 8
}
```
```

These GATES input (throws `Failure` if unmet); they do NOT downscale for detection.

****In `backend/config/models.py`:**

```
- **IndexingConfig** (line 180?273) ? **NO resolution/fps/proxy knobs**:
- `use_gpu: bool = True` (line 182)
- `detector_impl: str = "auto"` (line 188) ? selects runner (wsl/native/stub)
- `skip_ai_handoff: bool = False` (line 267) ? bypasses Gemini, keeps local rallies
- NO `target_fps`, `target_resolution`, `proxy_scale`, `downsample_width` fields exist

- **WasbIndexConfig** (line 68?143) ? trajectory detector config **only**:
- `stream_video: Optional[bool] = None` (line 112) ? FAST PATH: decode-on-fly vs PNG
extraction
- `batch_size: Optional[int] = None` (line 118)
- `prefetch_batches: Optional[int] = None` (line 121)
- **NO resolution/proxy fields**

- **IndexingConfig.ai_chunk_scale_width** (line 261):
  ```python
 ai_chunk_scale_width: int = 1280 # Width (px) Gemini chunks are re-encoded to
  ```

  This is **Gemini upload scaling** (AFTER detection), not input proxy. Default `0` = legacy
  stream-copy at source res` (line 259?260).
```

3. **DETECTION PIPELINE INSERTION POINT FOR GPU-FFMPEG PROXY**

****Current flow in `backend/orchestration.py:790?796`:**

```python

**Line 789: compute\_actual = "in\_process"**

segmenter = segmenter\_cls(db, config)

try:

```
 # Serialize the CPU/GPU-heavy in-process detection
 with _LOCAL_COMPUTE_LANE:
 result = segmenter.process_video(
 video_id, local_video, req.player_pool, req.max_frames
)
 ...
```

**\*\*Proposed insertion point (between line 788 and 791):\*\***

```python

NEW: GPU-ffmpeg proxy transcode BEFORE segmentation

if config.indexing.wasb.proxy_transcode_width > 0:

```
    local_video = _maybe_transcode_proxy(
        local_video, config.indexing.wasb
```


)
...

```
**Existing ffmpeg usage (for reference):**
- `backend/utils/video.py:51?108` ? `extract_video_chunk(..., scale_width: Optional[int] =
None)` already supports `-vf scale=WIDTH:-2` (line 81).
- `backend/utils/annotation_proxy.py:70?124` ? `make_annotation_proxy(src, dst, max_height)`
uses CPU libx264 (line 90, preset=veryfast).
- **GPU candidate:** `ffmpeg -i video.mp4 -vf
"hwupload=cuda_device=0,scale_cuda=w=1280:h=720,hwdownload" -c:v hevc_nvenc ...` (NVENC) or
`-c:v h264_nvenc` for H.264.

**Coordinate with `_LOCAL_COMPUTE_LANE` (line 794):** wrap the proxy step too, since it's
CPU/GPU-heavy.
```

4. ****REEL COMPILATION: TIMESTAMPS MAP CLEANLY BACK TO ORIGINAL****

****YES ? timestamps are preserved exactly, no proxy-aware mapping needed.****

****Flow:****

1. ****Detection on proxy**** (proposed): WASB detects rallies at proxy resolution (e.g., 720p).
2. ****Timestamps:**** `runner.trajectory\_to\_action\_windows(...)` (trajectory_hybrid.py:290?308) outputs `start` and `end` in ****seconds**** ? independent of frame resolution.

```
3. **Reel compilation** (orchestration.py:1025, compiler.py:382?413):
    ```python
 time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
 # ? VideoCompiler.compile_highlights(local_src, time_pairs, out_name)
 ...
```

```
4. **Extraction from ORIGINAL** (compiler.py:449?457):
    ```python
    clip_path, skip_record = self._trim_one_segment(
        idx, raw_start, raw_end, # seconds; resolution-agnostic
        ..., raw_video_path,     # ORIGINAL, not proxy
        ...
    )
    ...
```

```
5. **Frame-accurate trimming** (compiler.py:254?280): `extract_video_chunk(video_path,
start_time, duration, ...)` slices frames directly from the **original source**.
```

****Result:**** Timestamps detected on 720p30 proxy cut ****ORIGINAL 1080p60**** frames without remapping ? they're both motion-based timecode, not frame-count. Duration parity validated in annotation_proxy.py:112 proves this (src_dur ? proxy_dur after transcode).

5. ****PER-REQUEST OVERRIDE CAPABILITY****

****Current request model (`backend/main.py::ProcessRequest`, line 1158?1173 in job_worker.py context):****

```
```python
def _job_to_request(job: Dict[str, Any]):
 ...
 return _main.ProcessRequest(
 video_path=job["video_path"],
 player_pool=job.get("player_pool"),
 max_frames=job.get("max_frames"),
 segmenter=job.get("segmenter"),
 compute=job.get("compute"), # per-request cloud override
 force=bool(params.get("force")), # bypass cache
)
...
```
```

****Existing per-request knobs:****

- `compute="cloud"` (line 140) ? override server default
- `force=true` (line 141) ? skip cache, re-segment
- `segmenter="wasb\_hybrid"` (POST param at API submit)
- `max\_frames` (frame cap for testing)

****Proxy setting NOT yet present:****

No `skip\_ai\_handoff`, `timeout\_sec`, or `proxy\_resolution` overrides exist as request params. The `skip\_ai\_handoff` knob **exists in config only** (`indexing.skip\_ai\_handoff: bool = False`, models.py:267) for fully-local mode.

****To add proxy override:****

```
```python
ProcessRequest(
 ...,
 proxy_transcode_width: Optional[int] = None, # per-request override
)
```
```

Then in orchestration.py `_maybe_transcode_proxy`: use request value or fall back to config default.

6. ****SUMMARY TABLE: RESOLUTION/FPS CONFIG LANDSCAPE****

| Field | Location | Default | Purpose | Affects Detection Input? |
|------------------------------------|--------------------|---------------------|---|-------------------------------|
| `validation.min_resolution_width` | config.json:6 | 1280 | Input gate (reject < 1280) | ? |
| (validation only) | | | | |
| `validation.min_resolution_height` | config.json:7 | 720 | Input gate (reject < 720) | ? |
| (validation only) | | | | |
| `validation.min_fps` | config.json:8 | 29.9 | Input gate (reject < 29.9) | ? (validation only) |
| | | | | |
| `indexing.use_gpu` | config.json:27 | true | Enable GPU for segmenters | ? (detector backend) |
| | | | | |
| `indexing.detector_impl` | models.py:188 | "auto" | Runner: auto/stub/native | ? (runner selection) |
| | | | | |
| `indexing.wasb.stream_video` | models.py:112 | None (?True native) | Decode on-fly vs PNG cache | ~ (speed only, no res change) |
| | | | | |
| `indexing.ai_chunk_scale_width` | models.py:261 | 1280 | **POST-detection** Gemini upload scale | ? (downstream, not input) |
| | | | | |
| **Proxy/transcode** | **NOT YET** | **??** | **Would go in indexing.wasb** | **? (proposed)** |

7. ****KEY DISCOVERY: No Proxy Knob Exists Yet****

The `indexing.wasb` config block carries ****zero**** proxy/transcode directives. Issue #483's design calls for:

```
```
"indexing": {
 "wasb": {
 "proxy_transcode_enabled": bool, // proposed
 "proxy_target_resolution": "720p30", // proposed
 "proxy_fps_target": 30, // proposed (or infer from resolution)
 }
}
```
```

****Current code does NOT have these.**** The roadmap labels P7 #483 as future work, gated on golden-set quality validation.

8. ****TIMESTAMPS SURVIVAL ACROSS DETECTION: VERIFIED NO REMAP NEEDED****

****Fact:** Motion-based timestamps are resolution-agnostic.**

- `trajectory\_hybrid.py:290`: `runner.trajectory\_to\_action\_windows(points, fps=fps, frame\_width=frame\_width, ...)` ? the `fps` is video FPS (30 or 60), not changed by proxy.
- Window boundaries (start, end) are ****seconds of elapsed time****, not frame indices.
- `compiler.py:448?457`: iteration over `(raw\_start, raw\_end)` pairs directly, no rescaling.
- Extraction `extract\_video\_chunk(video\_path, start\_time, duration, ...)` ? `start\_time` (seconds) works identically on 1080p60 or 720p30.
- ****Validation:**** `annotation\_proxy.py:112` checks `abs(proxy\_dur - src\_dur) > 0.5s` and discards if mismatched, confirming duration parity proves timeline integrity.

****Conclusion:**** Rally timestamps detected on proxy ****directly apply to the original source**** with zero adjustment needed.

9. ****RECOMMENDED CONFIG SHAPE FOR #483 IMPLEMENTATION****

Add to `IndexingConfig.wasb` (models.py):

```
```python
```

### **GPU-transcode proxy for faster detection (#483)**

```
proxy_transcode_enabled: bool = False # default OFF until golden-gate approves
proxy_target_resolution: Literal["720p30", "720p60", "1080p30"] = "720p30"
proxy_device: str = "cuda" # "cuda" | "cpu" (default cuda for NVENC speed)
```
```

Then in orchestration.py, before line 795:

```
```python
if idx_cfg.wasb.proxy_transcode_enabled and seg_name.startswith("wasb"):
 local_video = _maybe_transcode_gpu_proxy(local_video, idx_cfg.wasb)
```
```

And in the ProcessRequest / job params, to allow per-request override:

```
```python
force_proxy_resolution: Optional[str] = None # e.g., "720p30"
```
```

FINAL ANSWER: All Five Questions

1. ****Existing proxy/transcode?**** ? No. #483 is designed but deferred to P7 (golden-gate pending).
2. ****Config fields?**** ? Validation gates exist (min_resolution_width/height, min_fps) but they REJECT, not downscale. No proxy/transcode knobs in indexing.wasb config yet.
3. ****Worker insertion point?**** ? Between orchestration.py line 788 and 791, after validator but before `segmenter.process\_video()`.
4. ****Timestamp remapping?**** ? None needed. Rally times are seconds, resolution-agnostic; reel cuts from ORIGINAL using the same timecodes detected on proxy.
5. ****Per-request overrides?**** Partially. `skip\_ai\_handoff`, `force`, `compute` exist; `timeout\_sec` and proxy config are config-only, not per-request yet.